République du Bénin

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Nationale des Sciences, Technologies, Ingénierie et Mathématiques d'Abomey (UNSTIM)

École Nationale Supérieure de Génie Mathématique et Modélisation (ENSGMM)

Implémentation du modèle d'infection par le VIH-SIDA

Modèles intra-hôte et de population : analyse mathématique, stabilité et résolution numérique avec BLAS, LAPACK et Intel MKL en Python

Équipe de projet – Groupe 5

ADOGOUN Déo Gracias Mahugnon

BIAHOU Ahossou Junior Marie-Clémenceau

SYLLA IMOROU Izou Dine

Promotion : ENSGMM1 • **UE :** Outils de calcul scientifique

Année académique : 2026–2027

Sous la direction de : Dr. Carlos AGOSSOU

Table des matières

Introduction **3**

1 Présentation d'Intel MKL (oneMKL) **4**
1.1 Qu'est-ce qu'Intel MKL ? . . . 4
1.2 Domaines couverts . . . 4
1.3 Avantages et alternatives du marché . . . 5

2 BLAS (Basic Linear Algebra Subprograms) **6**
2.1 Origine et philosophie de conception . . . 6
2.2 Les trois niveaux de BLAS et l'intensité arithmétique . . . 6
2.3 Conventions de nommage universelles . . . 7

3 LAPACK (Linear Algebra PACKage) **8**
3.1 Présentation et algorithmes par blocs . . . 8
3.2 Fonctionnalités et routines pilotes . . . 8
3.3 Schéma vectoriel de la pile logicielle . . . 8

4 Installation, configuration et utilisation **10**
4.1 Configuration sous Python et vérification du backend . . . 10
4.2 Appel direct de LAPACK depuis Python . . . 10

5 Le VIH-SIDA : contexte biologique et échelles **12**
5.1 Physiopathologie de l'infection . . . 12
5.2 Nécessité de deux échelles de modélisation . . . 12

6 Le modèle intra-hôte : formulation et analyse **13**
6.1 Formulation mathématique . . . 13
6.2 Positivité, bornitude et nombre de reproduction $\mathcal{R}_0$ . . . 14
6.3 Stabilité locale et globale . . . 16

7 Méthodes numériques et LAPACK **19**
7.1 Systèmes linéaires, erreur directe et conditionnement . . . 19
7.2 Équilibrage de la matrice jacobienne avec DGEEQU . . . 20
7.3 Méthode de Newton avec DGESV et calcul spectral DGEEV . . . 20

8 Intégration numérique des équations **22**
8.1 Raideur et instabilité des schémas explicites . . . 22
8.2 Schéma retenu et vérification . . . 23

9 Résultats et interprétation médicale **25**
9.1 Primo-infection et portrait de phase . . . 25
9.2 Efficacité thérapeutique et rebond viral . . . 25

10 Modèle épidémiologique de population : S, I_1, I_2 **28**
10.1 Formulation du modèle de population 28
10.2 Nombre de reproduction de base et équilibres 29
10.3 Scénarios d'intervention de santé publique 29

11 Benchmark haute performance BLAS / LAPACK **31**
11.1 Protocole de mesure 31
11.2 Résultats et analyse du gain de performance 31

Conclusion et perspectives **34**

Références bibliographiques **36**

A Code source du mini-projet (`vih_lapack.py`) **37**
A.1 Le module du modèle : `vih_lapack.py` 37
A.2 Le script de simulation : `simulation_intra.py` 39

B Code source du benchmark (`benchmark_blas.py`) **43**

C Guide d'exécution et reproductibilité **45**
C.1 Contenu du dossier 45
C.2 Étape 1 : préparer l'environnement Python 45
C.3 Étape 2 : vérifier la bibliothèque BLAS/LAPACK active 45
C.4 Étape 3 : tester le module du modèle 46
C.5 Étape 4 : reproduire les résultats numériques 46
C.6 Étape 5 : refaire le benchmark 46
C.7 Comparer avec Intel MKL 47
C.8 Étape 6 : compiler le rapport 47
C.9 Problèmes fréquents 47

Introduction

Résumé en clair : de quoi parle ce projet ?

Ce travail étudie la dynamique du virus de l'immunodéficience humaine (VIH) à deux échelles complémentaires :

1. **À l'échelle du patient (modèle intra-hôte) :** comment le virus infecte les lymphocytes T CD4+ du sang et comment un traitement antirétroviral bloque cette infection ;
2. **À l'échelle de la population (modèle épidémiologique) :** comment l'épidémie se propage entre personnes saines, personnes infectées sans symptômes et personnes au stade SIDA.

Pour résoudre ces équations de façon fiable et rapide, nous nous appuyons sur la pile logicielle standard du calcul scientifique : **Intel MKL**, qui fournit des implémentations optimisées de **BLAS** et de **LAPACK**, appelées depuis **Python**.

Pourquoi un tel sujet ? Un modèle de dynamique virale est un système d'équations différentielles non linéaires *raide* : certaines grandeurs évoluent en minutes (le virus libre), d'autres en mois (les lymphocytes). Sa résolution fiable demande des solveurs d'algèbre linéaire robustes (factorisation LU, valeurs propres, conditionnement) : c'est exactement le rôle de LAPACK, lui-même construit sur les opérations de base de BLAS.

Les quatre objectifs du travail :

1. Comprendre l'articulation de la pile logicielle **MKL / BLAS / LAPACK** et savoir la configurer et l'appeler depuis Python ;
2. Formuler et analyser mathématiquement le modèle intra-hôte (positivité, bornitude, nombre de reproduction $\mathcal{R}_0$, stabilité) et le modèle de population S, I_1, I_2 ;
3. Résoudre numériquement ces modèles avec des routines LAPACK (`DGESV`, `DGEEQU`, `DGEEV`), en maîtrisant la raideur grâce à un schéma d'Euler implicite, puis en interpréter les résultats sur le plan médical ;
4. Mesurer le gain de performance apporté par BLAS/LAPACK par rapport à un code Python naïf.

Organisation du rapport. Les chapitres 1 à 4 présentent les outils (MKL, BLAS, LAPACK, installation). Le chapitre 5 rappelle le contexte biologique. Les chapitres 6 à 9 traitent du modèle intra-hôte (analyse, méthodes numériques, intégration, résultats). Le chapitre 10 est consacré au modèle de population et le chapitre 11 au benchmark de performance. Le code complet figure en annexe.

Chapitre 1

Présentation d'Intel MKL (oneMKL)

1.1 Qu'est-ce qu'Intel MKL ?

Définition

Intel MKL (*Math Kernel Library*, aujourd'hui intégrée à la suite **Intel oneAPI** sous le nom **oneMKL**) est une bibliothèque de calcul numérique hautement optimisée pour les processeurs Intel. Elle fournit, entre autres, des implémentations de **BLAS** et de **LAPACK**.

Il faut bien distinguer trois notions qui sont souvent confondues :

- **BLAS** est une *spécification* : une liste standardisée d'opérations élémentaires d'algèbre linéaire (chapitre 2) ;
- **LAPACK** est une *bibliothèque algorithmique* qui résout de vrais problèmes (systèmes linéaires, valeurs propres, ...) en s'appuyant sur BLAS (chapitre 3) ;
- **Intel MKL** est une *implémentation industrielle optimisée* de ces deux interfaces, avec des noyaux de calcul adaptés au matériel.

Un programme écrit une seule fois contre l'interface standard (par exemple l'appel à `DGEMM`) peut ainsi être exécuté avec différentes implémentations, sans modification du code source.

1.2 Domaines couverts

Au-delà de BLAS et de LAPACK, oneMKL regroupe plusieurs familles de routines (tableau 1.1).

Table 1.1: Principaux domaines couverts par Intel oneMKL.

Domaine	Contenu
BLAS et LAPACK	Algèbre linéaire dense : produits matriciels, factorisations LU/QR/-Cholesky, valeurs propres, SVD.
Algèbre linéaire creuse	BLAS creux, solveurs directs (PARDISO) et itératifs.
Transformées de Fourier	FFT mono- et multidimensionnelles.
Fonctions vectorielles	Fonctions mathématiques appliquées à des vecteurs entiers (exponentielle, logarithme, ...).
Statistiques et aléatoire	Générateurs de nombres aléatoires, statistiques de base.

Dans ce projet, seuls BLAS et LAPACK sont utilisés : les produits matriciels (`DGEMM`), la résolution de systèmes (`DGESV`), l'équilibrage (`DGEEQU`), l'estimation du conditionnement (`DGECON`) et le calcul spectral (`DGEEV`).

1.3 Avantages et alternatives du marché

Les performances d'oneMKL reposent sur trois leviers matériels :

1. **La vectorisation SIMD** (AVX2, AVX-512) : une seule instruction traite plusieurs nombres flottants à la fois ;
2. **Le parallélisme multi-cœurs** (threads OpenMP) : les grandes matrices sont découpées en blocs répartis sur les cœurs ;
3. **Le choix dynamique du noyau** : au lancement, la bibliothèque détecte le modèle de processeur et sélectionne la version du code la mieux adaptée.

Le tableau 1.2 compare les principales implémentations disponibles.

Table 1.2: Comparaison des bibliothèques d'algèbre linéaire.

Bibliothèque	Statut / licence	Particularités
Netlib (référence)	Open source	Code de référence en Fortran, non optimisé ; sert d'étalon de comparaison.
OpenBLAS	Open source (BSD)	Rapide et multiplateforme ; c'est le backend livré par défaut avec les paquets NumPy/SciPy installés par `pip`.
BLIS	Open source (BSD)	Cadre modulaire fondé sur des micro-noyaux réutilisables.
Intel oneMKL	Gratuite, non libre	Très performante sur architectures x86-64 ; distribuée avec oneAPI et disponible dans l'écosystème `conda`.

Mini-résumé du chapitre 1 : Intel MKL

1. BLAS est une spécification, LAPACK une bibliothèque algorithmique, MKL une implémentation optimisée des deux.
2. MKL tire parti de la vectorisation, du multithreading et de la détection du processeur.
3. OpenBLAS, BLIS et Netlib sont des alternatives ; le code appelant reste identique.

Chapitre 2

BLAS (Basic Linear Algebra Subprograms)

2.1 Origine et philosophie de conception

BLAS (*Basic Linear Algebra Subprograms*) est née à la fin des années 1970 avec les routines vectorielles de Lawson, Hanson, Kincaid et Krogh [10]. L'idée directrice est de *séparer* les algorithmes de haut niveau (qui décrivent « quoi calculer ») des noyaux de bas niveau (qui décrivent « comment le calculer vite sur une machine donnée »). Les fabricants optimisent une fois les noyaux BLAS ; tous les programmes qui les appellent en bénéficient.

2.2 Les trois niveaux de BLAS et l'intensité arithmétique

BLAS est organisée en trois niveaux, selon la dimension des objets manipulés (tableau 2.1) ; le niveau 3 a été introduit en 1990 [11].

Table 2.1: Les trois niveaux de BLAS et leur intensité arithmétique.

Niveau	Opérations	Données	Calculs	Intensité	Exemple
1	Vecteur – vecteur	$O(n)$	$O(n)$	$O(1)$ (faible)	`DAXPY` : $y \leftarrow \alpha x + y$
2	Matrice – vecteur	$O(n^2)$	$O(n^2)$	$O(1)$ (faible)	`DGEMV` : $y \leftarrow \alpha Ax + \beta y$
3	Matrice – matrice	$O(n^2)$	$O(n^3)$	$\mathbf{O(n)}$ (élevée)	`DGEMM` : $C \leftarrow \alpha AB + \beta C$

Intensité arithmétique et mémoire cache

- **Mémoire cache :** petite mémoire très rapide située au cœur du processeur (niveaux L1, L2, L3), bien plus rapide que la mémoire vive (RAM) mais de taille réduite.
- **Intensité arithmétique :** nombre d'opérations flottantes effectuées par donnée transférée depuis la RAM.

Pourquoi le niveau 3 est-il le plus efficace ?

- Aux **niveaux 1 et 2**, chaque nombre lu en RAM n'est utilisé que pour $O(1)$ opérations : le processeur attend les données ; on dit que le calcul est *limité par la mémoire*.
- Au **niveau 3**, multiplier deux matrices $n \times n$ demande $O(n^2)$ données mais $2n^3$ opérations. En découpant les matrices en blocs qui tiennent dans le cache, chaque donnée est réutilisée de nombreuses fois [12] : le calcul devient *limité par la puissance du processeur*.

2.3 Conventions de nommage universelles

Le nom d'une routine BLAS (ou LAPACK) encode ce qu'elle fait :

`[Précision][Type de matrice][Opération]`

- **Précision :** `S` (réel simple, 32 bits), `D` (réel double, 64 bits, standard en science), `C` (complexe simple), `Z` (complexe double) ;
- **Type de matrice :** `GE` (générale), `SY` (symétrique), `TR` (triangulaire), …;
- **Opération :** `MM` (matrice–matrice), `MV` (matrice–vecteur), `SV` (résolution de système), `EV` (valeurs propres), …

Exemple : `DGEMM` = **D**ouble précision, matrice **GE**nérale, produit **M**atrice–**M**atrice.

Mini-résumé du chapitre 2 : BLAS

1. BLAS standardise les opérations élémentaires d'algèbre linéaire.
2. Niveau 1 : vecteurs ; niveau 2 : matrice–vecteur ; niveau 3 : matrice–matrice.
3. Seul le niveau 3 réutilise les données en cache et approche la puissance maximale du processeur.
4. Les noms de routines suivent une convention régulière (`DGEMM`, `DGESV`, …).

Chapitre 3

LAPACK (Linear Algebra PACKage)

3.1 Présentation et algorithmes par blocs

LAPACK (*Linear Algebra PACKage*) résout les problèmes courants de l'algèbre linéaire numérique : systèmes linéaires $Ax = b$, problèmes de moindres carrés, valeurs et vecteurs propres, décompositions en valeurs singulières [9]. Elle a succédé au début des années 1990 aux bibliothèques LINPACK et EISPACK.

Sa conception repose sur les **algorithmes par blocs** : au lieu de factoriser une matrice ligne par ligne (opérations de niveau 2, limitées par la mémoire), LAPACK la découpe en blocs rectangulaires et délègue l'essentiel du travail à `DGEMM` (niveau 3). Les performances de LAPACK dépendent donc directement de celles de l'implémentation BLAS sous-jacente, d'où l'importance de MKL ou d'OpenBLAS.

Notions clés

- **Factorisation LU avec pivotage partiel :** $PA = LU$, où P est une matrice de permutation, L triangulaire inférieure à diagonale unité et U triangulaire supérieure. Résoudre $Ax = b$ revient à résoudre $Ly = Pb$ (descente) puis $Ux = y$ (remontée).
- **Conditionnement** $\kappa(A) = \|A\|\,\|A^{-1}\| \geq 1$: il mesure la sensibilité de la solution aux perturbations des données. Une erreur relative de l'ordre de $\kappa(A)\,\varepsilon_{\text{mach}}$ est attendue, avec $\varepsilon_{\text{mach}} \approx 1{,}1 \times 10^{-16}$ en double précision.
- **Valeurs propres** λ : scalaires tels que $Av = \lambda v$ pour un vecteur $v \neq 0$; leurs parties réelles déterminent la stabilité d'un équilibre.

3.2 Fonctionnalités et routines pilotes

Les *routines pilotes* (*drivers*) résolvent un problème complet en un seul appel. Le tableau 3.1 regroupe celles utilisées dans ce projet.

3.3 Schéma vectoriel de la pile logicielle

La figure 3.1 montre comment les différents étages s'articulent, du modèle biologique jusqu'au processeur.

Table 3.1: Routines LAPACK exploitées dans le projet.

Problème	**Routine**	**Méthode**	**Coût**
Résoudre $Ax = b$	`DGESV`	Factorisation LU avec pivotage partiel	$\frac{2}{3}n^3$
Factorisation / substitution	`DGETRF` / `DGETRS`	LU, puis résolutions triangulaires	$\frac{2}{3}n^3$ / $O(n^2)$
Équilibrer une matrice	`DGEEQU`	Coefficients de mise à l'échelle des lignes et colonnes	$O(n^2)$
Estimer le conditionnement	`DGECON`	Estimation de $\|A^{-1}\|_1$ sans inverser A	$O(n^2)$
Valeurs propres générales	`DGEEV`	Réduction de Hessenberg et algorithme QR	$O(n^3)$
Valeurs singulières	`DGESDD`	SVD par diviser pour régner	$O(n^3)$
Symétrique : valeurs propres	`DSYEVD`	Diviser pour régner (matrices symétriques)	$O(n^3)$

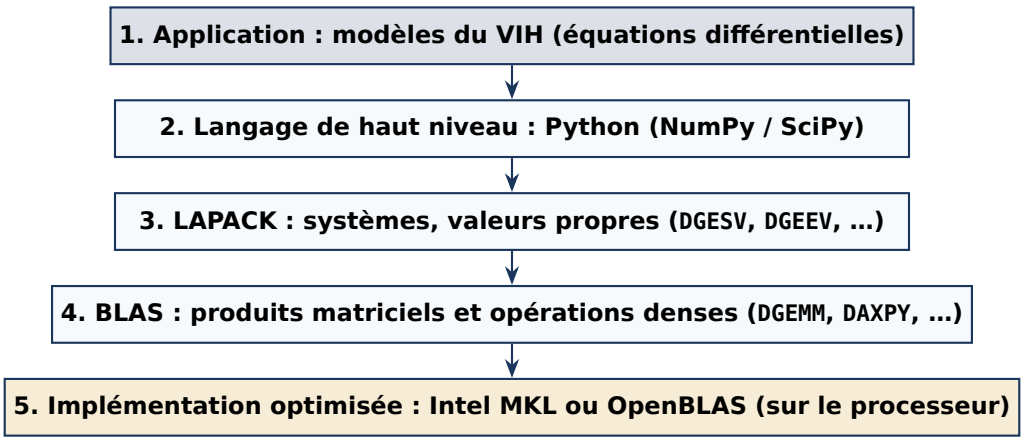


Figure 3.1: Architecture en couches : de la modélisation biologique jusqu'au processeur.

Mini-résumé du chapitre 3 : LAPACK

1. LAPACK résout les problèmes d'algèbre linéaire (systèmes, valeurs propres, SVD).
2. Elle s'appuie sur BLAS niveau 3 via des algorithmes par blocs.
3. `DGESV` (LU), `DGEEQU` (équilibrage), `DGECON` (conditionnement) et `DGEEV` (spectre) sont les routines centrales du projet.

Chapitre 4

Installation, configuration et utilisation

4.1 Configuration sous Python et vérification du backend

NumPy et SciPy sont précompilés avec une bibliothèque BLAS/LAPACK : généralement OpenBLAS pour les paquets installés par `pip`, éventuellement MKL dans une distribution `conda` configurée en ce sens. Le backend réellement actif se vérifie ainsi :

```python
import numpy as np
from threadpoolctl import threadpool_info

np.show_config()            # configuration BLAS/LAPACK vue par NumPy
for lib in threadpool_info():
    print(lib["internal_api"], lib["version"], lib["num_threads"])
```

Explications.

- `np.show_config()` affiche (sans valeur de retour) les bibliothèques avec lesquelles NumPy a été construit ;
- `threadpool_info()` interroge les bibliothèques réellement chargées en mémoire : nom (`openblas` ou `mkl`), version et nombre de threads.

Le nombre de threads se contrôle par les variables d'environnement `OMP_NUM_THREADS` ou `MKL_NUM_THREADS`, ou dans le code par `threadpoolctl.threadpool_limits`. Pour obtenir MKL, on crée par exemple un environnement `conda` dédié et on y installe NumPy/SciPy avec le backend MKL.

4.2 Appel direct de LAPACK depuis Python

Plutôt que la fonction générique `np.linalg.solve`, le module `scipy.linalg.lapack` expose les routines LAPACK d'origine :

```python
import numpy as np
from scipy.linalg import lapack

# Matrice en ordre Fortran (colonnes) et second membre
A = np.asfortranarray([[3.0, 1.0], [1.0, 2.0]])
b = np.array([9.0, 8.0])

# 1. Resolution de Ax = b par DGESV (LU avec pivotage partiel)
lu, piv, x, info = lapack.dgesv(A, b)
assert info == 0, "matrice singuliere"
print("x =", x)                     # [2. 3.]
```

```
12
13 # 2. Valeurs propres par DGEEV
14 wr, wi, vl, vr, info = lapack.dgeev(A)
15 print("valeurs propres =", wr + 1j * wi)
```

Explications.

- `np.asfortranarray` stocke la matrice par colonnes, convention de LAPACK, ce qui évite des copies ;
- `dgesv` renvoie les facteurs `lu`, les pivots `piv`, la solution `x` et un code d'erreur `info` (0 si tout va bien) ;
- `dgeev` renvoie les parties réelle `wr` et imaginaire `wi` des valeurs propres.

Mini-résumé du chapitre 4 : utilisation en Python

1. NumPy/SciPy se lient automatiquement à une bibliothèque BLAS/LAPACK ; `threadpoolctl` permet de l'identifier.
2. `scipy.linalg.lapack` donne un accès direct aux routines Fortran.
3. Toujours tester le code de retour `info`.

Chapitre 5

Le VIH-SIDA : contexte biologique et échelles

5.1 Physiopathologie de l'infection

Le VIH est un rétrovirus qui cible principalement les **lymphocytes T CD4+**, cellules qui coordonnent la réponse immunitaire. Le cycle d'infection est le suivant :

- un virion libre pénètre dans une cellule saine T ;
- il convertit son ARN en ADN et détourne la cellule pour produire de nouveaux virions ;
- la cellule infectée meurt en libérant ces virions, qui infectent à leur tour d'autres cellules.

Chez un patient non traité, on distingue trois phases :

1. **Phase aiguë (primo-infection) :** multiplication virale explosive, avec un pic de charge virale et une chute transitoire des CD4+ ;
2. **Phase chronique (plusieurs années) :** un régime quasi stationnaire (*set-point*) s'installe, sans symptômes mais avec destruction lente des CD4+ ;
3. **Stade SIDA :** sous le seuil clinique d'environ 200 CD4+/μL, des maladies opportunistes apparaissent.

5.2 Nécessité de deux échelles de modélisation

- **Échelle intra-hôte (jours, semaines) :** décrire la dynamique virale dans l'organisme d'un patient et évaluer l'effet d'un traitement antirétroviral ;
- **Échelle de la population (années, décennies) :** décrire la transmission entre individus et évaluer des politiques de santé publique (prévention, traitement).

Les deux échelles mettent en jeu des paramètres et des équations différents ; elles sont donc traitées séparément (chapitres 6–9 puis chapitre 10).

Mini-résumé du chapitre 5

1. Le VIH détruit les lymphocytes T CD4+ ; sans traitement, l'infection évolue vers le SIDA.
2. Deux échelles de temps coexistent : celle du patient (jours) et celle de la société (années).

Chapitre 6

Le modèle intra-hôte : formulation et analyse

6.1 Formulation mathématique

Le modèle de base de la dynamique virale [1, 2] comporte trois variables : $T(t)$ (CD4+ sains), $I(t)$ (cellules infectées productrices de virus), $V(t)$ (virions libres), toutes en concentration par μL de sang. L'effet d'un traitement antirétroviral d'efficacité $\eta \in [0,1]$ est de réduire le taux d'infection d'un facteur $(1-\eta)$:

$$\begin{cases} \dfrac{dT}{dt} = s - dT - (1-\eta)\beta TV \\ \dfrac{dI}{dt} = (1-\eta)\beta TV - \delta I \\ \dfrac{dV}{dt} = pI - cV \end{cases} \tag{6.1}$$

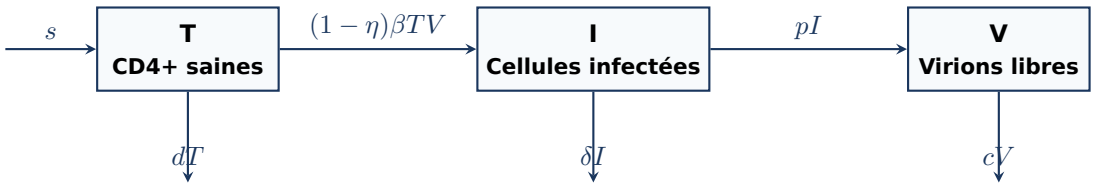


Figure 6.1: Schéma de transfert compartimental du modèle intra-hôte.

Table 6.1: Paramètres du modèle intra-hôte (valeurs de `vih_lapack.py`).

Param.	Signification	Valeur	Unité	Interprétation
s	Production de CD4+	10	cellules/μL/j	Source constante
d	Mortalité des CD4+	$0{,}01$	j^{-1}	Durée de vie moyenne $1/d = 100$ j
β	Infectivité	$4{,}6 \times 10^{-5}$	μL/virion/j	Taux de contact efficace
δ	Mortalité des infectées	$0{,}50$	j^{-1}	Durée de vie $1/\delta = 2$ j
p	Production virale	1000	virions/cell./j	Virions libérés par cellule infectée
c	Clairance virale	23	j^{-1}	Demi-vie $\ln 2/c \approx 43$ min
η	Efficacité du traitement	$[0,1]$	–	$\eta = 0$: aucun traitement

6.2 Positivité, bornitude et nombre de reproduction $\mathcal{R}_0$

Le modèle est-il réaliste ? Positivité et bornitude

Avant d'étudier un modèle, on vérifie qu'il ne produit pas de résultats absurdes. Deux questions simples se posent.

1. Les concentrations restent-elles positives ? Une concentration de cellules ne peut pas être négative. On le vérifie en regardant ce qui se passe quand une variable touche 0 :

- si $T = 0$, alors $\dfrac{dT}{dt} = s > 0$: la moelle osseuse fabrique de nouvelles cellules, donc T remonte ;
- si $I = 0$, alors $\dfrac{dI}{dt} = (1-\eta)\beta TV \geq 0$: les infections ne peuvent qu'ajouter des cellules infectées ;
- si $V = 0$, alors $\dfrac{dV}{dt} = pI \geq 0$: les cellules infectées ne font qu'ajouter des virions.

Une variable qui arrive à 0 ne peut donc jamais devenir négative : **le modèle est cohérent avec la réalité**.

2. Les concentrations restent-elles bornées ? Rien ne doit « exploser » vers l'infini.

- Les cellules (T et I ensemble) gagnent au plus $s = 10$ par jour et en perdent au moins une fraction $d = 1\,\%$ par jour. Leur total ne peut donc pas dépasser $s/d = 1000$ cellules/μL.
- Les virions sont produits par les cellules infectées (au plus p virions par cellule et par jour) et éliminés au taux c. La charge virale ne peut donc pas dépasser environ $p \times 1000/c \approx 4{,}3 \times 10^4$ virions/μL.

Le pic de virémie observé en primo-infection ($\approx 1{,}7 \times 10^4$) est bien sous cette limite.

Le nombre de reproduction de base $\mathcal{R}_0$

Définition de $\mathcal{R}_0$

Le **nombre de reproduction de base** $\mathcal{R}_0$ est le nombre moyen de *nouvelles* cellules infectées produites par *une seule* cellule infectée introduite dans un organisme entièrement sain.

- $\mathcal{R}_0 < 1$: chaque cellule infectée en remplace moins d'une ; l'infection s'éteint ;
- $\mathcal{R}_0 > 1$: chaque cellule infectée en produit plus d'une ; l'infection s'installe.

Pour calculer $\mathcal{R}_0$ de façon systématique, on utilise la **méthode de la matrice de prochaine génération** [4, 5]. L'idée est de suivre uniquement les variables « infectées » (ici I et V) et de répondre à la question : *combien de nouvelles infections chaque infecté engendre-t-il au cours de sa vie ?*

Étape 1 : l'état sain. Sans virus, les CD4+ sont à leur niveau naturel $E_0 = (T_0, 0, 0)$ avec $T_0 = s/d = 10/0{,}01 = 1000$ cellules/μL. C'est l'état de départ de l'étude, appelé *équilibre sans infection*.

Étape 2 : séparer deux types de phénomènes. Pour les variables infectées $y = (I, V)^\top$, on écrit $\dfrac{dy}{dt} = \mathcal{F}(y) - \mathcal{V}(y)$, où :

- $\mathcal{F}$ (« F cursif ») regroupe les **nouvelles infections**, c'est-à-dire ce qui *fait apparaître* de nouveaux infectés ;
- $\mathcal{V}$ (« V cursif ») regroupe tout **le reste** : les morts, les disparitions et les transferts d'un compartiment à l'autre (cellule infectée qui produit du virus, virus qui est éliminé).

Chaque vecteur a une ligne par variable infectée (première ligne : I, deuxième ligne : V) :

$$\mathcal{F} = \begin{pmatrix} (1-\eta)\beta T V \\ 0 \end{pmatrix} \begin{matrix} \leftarrow \text{ ligne de } I \\ \leftarrow \text{ ligne de } V \end{matrix} \qquad \mathcal{V} = \begin{pmatrix} \delta I \\ -pI + cV \end{pmatrix}.$$

- $\mathcal{F}_1 = (1-\eta)\beta T V$: les virions infectent de nouvelles cellules T, qui deviennent des cellules infectées I. C'est la *seule* vraie nouvelle infection. On met 0 à la ligne de V, car les virions ne naissent pas d'une infection directe mais de la production pI, qui est un transfert.
- $\mathcal{V}_1 = \delta I$: les cellules infectées meurent au taux δ.
- $\mathcal{V}_2 = -pI + cV$: le virus est éliminé au taux c (terme $+cV$) et produit par les cellules infectées (terme $-pI$, le signe $-$ vient du fait que $\mathcal{V}$ est soustrait).

Étape 3 : les matrices F et V_m. On cherche maintenant la *vitesse* avec laquelle chaque quantité réagit à un petit nombre d'infectés. Pour cela, on prend la **matrice jacobienne**, c'est-à-dire la matrice des **dérivées partielles premières** : la case de la ligne i et de la colonne j vaut la dérivée de la ligne i du vecteur par rapport à la j-ième variable infectée (I pour la colonne 1, V pour la colonne 2), calculée à l'état sain $T = T_0$:

$$F_{ij} = \frac{\partial \mathcal{F}_i}{\partial y_j}, \qquad (V_m)_{ij} = \frac{\partial \mathcal{V}_i}{\partial y_j}.$$

Le vecteur $\mathcal{F}$ a une colonne, mais F en a **deux**, car on dérive chaque ligne par rapport aux **deux** variables I et V. Le calcul donne :

$$F = \begin{pmatrix} 0 & (1-\eta)\beta T_0 \\ 0 & 0 \end{pmatrix}, \qquad V_m = \begin{pmatrix} \delta & 0 \\ -p & c \end{pmatrix}.$$

- Dans F, le seul coefficient non nul (ligne I, colonne V) dit : *chaque virion crée de nouvelles cellules infectées au taux* $(1-\eta)\beta T_0$. Toutes les autres cases sont nulles : I ne dépend pas de I dans $\mathcal{F}_1$, et la ligne de V est nulle.
- V_m décrit les départs : δ (mort d'une cellule infectée), c (élimination d'un virion) et $-p$ (une cellule infectée alimente le compartiment des virions).

Comme V_m est triangulaire, on l'inverse facilement :

$$V_m^{-1} = \begin{pmatrix} \dfrac{1}{\delta} & 0 \\ \dfrac{p}{\delta c} & \dfrac{1}{c} \end{pmatrix}.$$

Ses coefficients sont des *durées* : $1/\delta$ est la durée de vie d'une cellule infectée (2 jours), $1/c$ celle d'un virion (environ 1 heure), et $p/(\delta c)$ le nombre de virions présents « au cours de la vie » d'une cellule infectée.

Étape 4 : la matrice de prochaine génération $K = F\,V_m^{-1}$. En multipliant le « taux de nouvelles infections » (F) par les « durées de vie » (V_m^{-1}), on obtient le *nombre* de nouvelles infections créées pendant toute la vie d'un infecté :

$$K = F\,V_m^{-1} = \begin{pmatrix} \dfrac{(1-\eta)\beta p T_0}{\delta c} & \dfrac{(1-\eta)\beta T_0}{c} \\ 0 & 0 \end{pmatrix}.$$

- $K_{11} = \dfrac{(1-\eta)\beta p T_0}{\delta c}$: nombre de nouvelles cellules infectées créées, via ses virions, par **une cellule infectée** ;
- $K_{12} = \dfrac{(1-\eta)\beta T_0}{c}$: nombre de nouvelles cellules infectées créées par **un virion** ;
- la deuxième ligne est nulle : une infection ne crée jamais directement un virion, uniquement une cellule infectée.

Le nombre cherché $\mathcal{R}_0$ est la **plus grande valeur propre** de K (le *rayon spectral*). La matrice étant triangulaire, ses valeurs propres sont lues sur la diagonale : K_{11} et 0. On obtient donc

$$\mathcal{R}_0(\eta) = \frac{(1-\eta)\,\beta\,p\,s}{d\,\delta\,c} = \underbrace{\frac{p}{\delta}}_{\substack{\text{virions produits par} \\ \text{une cellule infectée}}} \times \underbrace{\frac{(1-\eta)\beta T_0}{c}}_{\substack{\text{cellules infectées par} \\ \text{un virion}}}. \tag{6.2}$$

Cette forme se lit facilement : une cellule infectée libère $p/\delta = 2000$ virions pendant sa vie, et chaque virion, avant d'être éliminé, infecte en moyenne $(1-\eta)\beta T_0/c \approx 0{,}002$ cellule.

Étape 5 : application numérique. Sans traitement ($\eta = 0$) :

$$\mathcal{R}_0(0) = \frac{4{,}6 \times 10^{-5} \times 1000 \times 1000}{0{,}5 \times 23} = \frac{46}{11{,}5} = 4{,}0.$$

Une cellule infectée en produit donc 4 nouvelles : l'infection s'installe. Pour la faire disparaître, il faut $\mathcal{R}_0(\eta) = (1-\eta)\mathcal{R}_0(0) < 1$, soit

$$\eta > \eta_c = 1 - \frac{1}{\mathcal{R}_0(0)} = 1 - \frac{1}{4} = 0{,}75.$$

Conclusion médicale : un traitement doit bloquer au moins $75\,\%$ des nouvelles infections pour que le virus disparaisse ; à $\eta = 70\,\%$, $\mathcal{R}_0 = 1{,}2 > 1$ et l'infection persiste.

6.3 Stabilité locale et globale

Qu'est-ce que la stabilité ?

Un **équilibre** est un état où plus rien ne change ($\frac{dT}{dt} = \frac{dI}{dt} = \frac{dV}{dt} = 0$). Il est **stable** si, lorsqu'on s'en écarte un peu, le système y *revient* (comme une bille au fond d'une cuvette), et **instable** s'il s'en éloigne (comme une bille posée sur le sommet d'une bosse). Pour le savoir, on calcule les **valeurs propres** de la jacobienne (matrice des dérivées premières du système complet) à l'équilibre : si toutes ont une **partie réelle négative**, l'équilibre est stable ; si l'une d'elles a une partie réelle positive, il est instable. Une valeur propre complexe signifie des **oscillations**.

Les deux équilibres

- **Équilibre sans infection** $E_0 = (1000, 0, 0)$: le patient est sain. Il existe toujours.
- **Équilibre endémique** $E^* = (T^*, I^*, V^*)$: le virus est présent en permanence. Il existe si $\mathcal{R}_0 > 1$:

$$T^* = \frac{T_0}{\mathcal{R}_0}, \qquad I^* = \frac{s - dT^*}{\delta}, \qquad V^* = \frac{p}{c}\, I^*.$$

  Pour $\eta = 0$: $T^* = 250$, $I^* = 15$, $V^* = \frac{1000}{23} \times 15 \approx 652{,}2$.

Stabilité de l'équilibre sans infection

En posant $b = (1-\eta)\beta$, la jacobienne en E_0 est

$$J(E_0) = \begin{pmatrix} -d & 0 & -bT_0 \\ 0 & -\delta & bT_0 \\ 0 & p & -c \end{pmatrix}.$$

Une de ses valeurs propres est $-d < 0$ (toujours stable). Les deux autres viennent du petit bloc $\begin{pmatrix} -\delta & bT_0 \\ p & -c \end{pmatrix}$: leur somme est $-(\delta + c) < 0$ et leur produit vaut $\delta c - bpT_0 = \delta c\,(1 - \mathcal{R}_0)$. Les deux valeurs propres sont donc négatives *exactement quand* $\mathcal{R}_0 < 1$. On en conclut :

- **si** $\mathcal{R}_0 < 1$: l'état sain est stable, le virus s'éteint ;
- **si** $\mathcal{R}_0 > 1$: l'état sain est instable, une trace de virus suffit à déclencher l'infection.

Pour $\mathcal{R}_0 \leq 1$, on peut même montrer que le virus disparaît *quel que soit l'état de départ* du patient (stabilité globale) ; ce résultat classique, démontré dans la littérature [6, 7], n'est pas redémontré ici.

Stabilité de l'équilibre endémique

Pour $\eta = 0$ ($\mathcal{R}_0 = 4$), le calcul numérique des valeurs propres de $J(E^*)$ par `DGEEV` donne

$$\lambda_1 = -23{,}50 \text{ j}^{-1}, \qquad \lambda_{2,3} = -0{,}0197 \pm 0{,}1196\, i \text{ j}^{-1}.$$

Les trois parties réelles sont négatives : E^* **est stable**.

- λ_1, très négative, correspond à l'élimination *rapide* du virus libre (en environ $1/23$ de jour) ;
- la paire complexe $-0{,}0197 \pm 0{,}1196\, i$ correspond à des **oscillations amorties** : la partie imaginaire donne la période $2\pi/0{,}1196 \approx 53$ jours, et la partie réelle négative dit que l'amplitude diminue (d'un facteur e environ tous les $1/0{,}0197 \approx 51$ jours).

Ces valeurs propres expliquent les oscillations vues en primo-infection (chapitre 9). À $\mathcal{R}_0 = 1$, les deux équilibres échangent leur stabilité : c'est une **bifurcation transcritique** (figure 9.5). En dessous de 1, le patient guérit ; au-dessus, il reste infecté.

Mini-résumé du chapitre 6 : modèle intra-hôte

1. Le modèle (T, I, V) est cohérent : les concentrations restent positives et bornées.
2. $\mathcal{R}_0(\eta) = \dfrac{(1-\eta)\beta p s}{d\delta c}$ est le nombre de nouvelles cellules infectées par cellule infectée ; sans traitement, $\mathcal{R}_0 = 4$.

3. Méthode : $\mathcal{F}$ (nouvelles infections) et $\mathcal{V}$ (le reste) $\rightarrow$ jacobiennes F et V_m (dérivées premières) $\rightarrow K = FV_m^{-1} \rightarrow \mathcal{R}_0 =$ plus grande valeur propre de K.
4. L'efficacité critique est $\eta_c = 75\,\%$: au-delà, le virus s'éteint.
5. E_0 est stable si $\mathcal{R}_0 < 1$; E^* est stable si $\mathcal{R}_0 > 1$, avec des oscillations amorties de période environ 53 jours.

Chapitre 7

Méthodes numériques et LAPACK

7.1 Systèmes linéaires, erreur directe et conditionnement

La routine `DGESV` résout $Ax = b$ par factorisation LU avec pivotage partiel. Cet algorithme est *rétrograde-stable* en pratique [14] : la solution calculée $\hat{x}$ vérifie $(A+\Delta A)\hat{x} = b$ avec $\|\Delta A\|/\|A\|$ de l'ordre de la précision machine. Il en découle deux grandeurs à distinguer :

- le **résidu relatif** $\|b - A\hat{x}\|/(\|A\|\|\hat{x}\|)$, qui reste de l'ordre de 10^{-16} ;
- l'**erreur directe** $\|\hat{x} - x\|/\|x\|$, bornée par $\kappa(A)\,\varepsilon_{\text{mach}}$, et donc grande si A est mal conditionnée.

La figure 7.1 illustre ce contraste sur les matrices de Hilbert $H_{ij} = 1/(i + j - 1)$, notoirement mal conditionnées. Le conditionnement κ_1 est estimé sans inversion par `DGETRF` suivi de `DGECON` (fonction `cond1` du code).

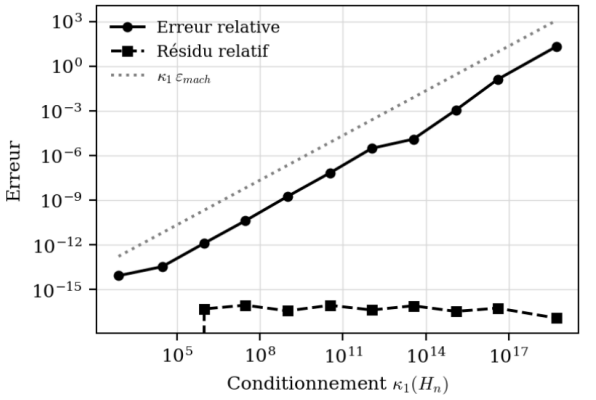


Figure 7.1: Matrices de Hilbert et `DGESV` : erreur et résidu en fonction de $\kappa_1(H_n)$.

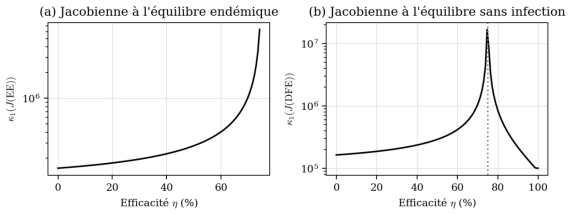


Figure 7.2: Conditionnement κ_1 de la jacobienne en fonction de η.

Lecture de la figure 7.1. L'erreur directe suit la droite $\kappa_1\varepsilon_{\text{mach}}$ lorsque κ_1 croît, tandis que le résidu reste au niveau de la précision machine : `DGESV` fournit toujours une solution qui satisfait presque exactement le système, mais ne peut pas compenser un mauvais conditionnement du problème lui-même.

Lecture de la figure 7.2. La jacobienne du modèle est mal conditionnée à cause de la disparité des échelles ($T \sim 10^3$ contre $V \sim 10^2$–10^4, $p = 1000$, $c = 23$) : $\kappa_1(J(E^*)) \approx 1{,}5 \times 10^5$ pour $\eta = 0$, et le conditionnement diverge lorsque l'on s'approche d'un point de bifurcation ($\eta \to \eta_c$).

7.2 Équilibrage de la matrice jacobienne avec DGEEQU

La routine `DGEEQU` calcule des coefficients diagonaux R (lignes) et C (colonnes) tels que $\widehat{J} = RJC$ ait des lignes et colonnes de normes comparables. Appliquée à la jacobienne du modèle :

Point	$\kappa_1(J)$	$\kappa_1(RJC)$
Équilibre endémique E^* ($\eta = 0$)	$1{,}5 \times 10^5$	≈ 69
Équilibre sain E_0 ($\eta = 0$)	$1{,}6 \times 10^5$	≈ 61
Pic de virémie (état $(60, 300, 17000)$)	$8{,}9 \times 10^4$	≈ 8

L'équilibrage réduit donc le conditionnement de trois à quatre ordres de grandeur : il est indispensable avant toute inversion ou résolution quand les variables ont des unités et des échelles très différentes.

7.3 Méthode de Newton avec DGESV et calcul spectral DGEEV

Pour calculer un équilibre $f(y^*) = 0$, on utilise la méthode de Newton :

$$J(y_k)\,\delta_k = -f(y_k), \qquad y_{k+1} = y_k + \delta_k,$$

où le système 3×3 est résolu à chaque itération par `DGESV` (fonction `newton_h`).

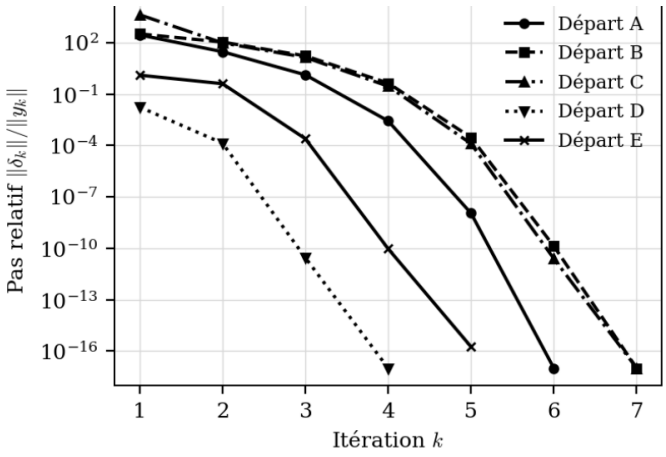


Figure 7.3: Convergence quadratique de la méthode de Newton (cinq points de départ).

Lecture de la figure 7.3. L'axe vertical (échelle logarithmique) donne le pas relatif $\|\delta_k\|/\|y_k\|$. Depuis chacun des points de départ, la méthode converge en moins de dix itérations, avec la chute de plus en plus rapide du pas caractéristique de la convergence quadratique.

Au point E^*, `DGEEV` fournit le spectre de la jacobienne donné au chapitre 6 : $\lambda_1 = -23{,}50$ et $\lambda_{2,3} = -0{,}0197 \pm 0{,}1196\,i$. Le rapport $|\lambda_{\text{max}}|/|\lambda_{\text{min}}| \approx 23{,}5/0{,}0197 \approx 1200$ quantifie la *raideur* du système, étudiée au chapitre 8.

Mini-résumé du chapitre 7 : solveurs LAPACK

1. `DGESV` donne un résidu au niveau de la précision machine ; l'erreur directe est limitée par $\kappa\,\varepsilon_{\text{mach}}$.
2. `DGEEQU` ramène $\kappa_1(J)$ d'environ 10^5 à quelques dizaines.
3. Newton + `DGESV` converge quadratiquement vers l'équilibre ; `DGEEV` en donne la

stabilité.

Chapitre 8

Intégration numérique des équations

8.1 Raideur et instabilité des schémas explicites

Raideur (*stiffness*)

Un système différentiel est dit **raide** lorsqu'il fait coexister des phénomènes d'échelles de temps très différentes. Ici : la clairance virale ($c = 23$ j^{-1}, demi-vie de 43 min) et la dynamique lente des CD4+ (échelle de $1/d = 100$ j). Le rapport des valeurs propres extrêmes vaut environ 1200.

Pour le schéma d'**Euler explicite** $y_{k+1} = y_k + hf(y_k)$, la stabilité sur la composante rapide impose

$$h < \frac{2}{|\lambda_{\max}|} \approx \frac{2}{23{,}5} \approx 0{,}085 \text{ jour } (\approx 2 \text{ heures}).$$

Au-delà, les erreurs sont amplifiées à chaque pas : avec $h = 0{,}1$ jour, notre simulation explicite déborde (valeurs non finies, dépassement de capacité des flottants) sur 30 jours simulés, alors que le phénomène physique est parfaitement régulier (figure 8.1).

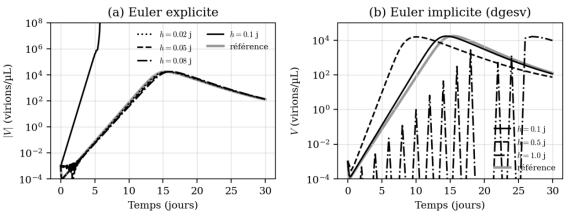


Figure 8.1: Euler explicite (a) et Euler implicite (b) pour plusieurs pas h.

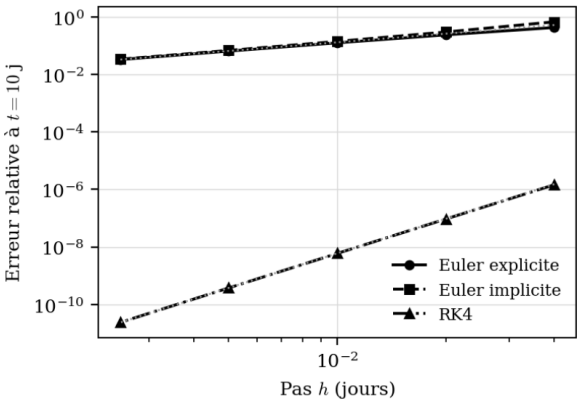


Figure 8.2: Erreur à $t = 10$ j en fonction du pas h.

8.2 Schéma retenu et vérification

Le schéma d'Euler implicite linéarisé

Pour s'affranchir de la contrainte de stabilité, nous retenons l'**Euler implicite** $y_{k+1} = y_k + hf(y_{k+1})$, linéarisé autour de y_k (une itération de Newton par pas) :

$$\big[I - h\,J(y_k)\big]\,\Delta y_k = h\,f(y_k), \qquad y_{k+1} = y_k + \Delta y_k. \tag{8.1}$$

À chaque pas, le système linéaire 3×3 est résolu par `DGESV`. Sur un problème linéaire $\dot{y} = \lambda y$, ce schéma est A-stable : le facteur d'amplification $1/(1 - h\lambda)$ est de module inférieur à 1 pour tout $h > 0$ et $\mathrm{Re}\,\lambda < 0$, quelle que soit la raideur. Il est d'ordre 1. Une projection sur l'orthant positif ($y \leftarrow \max(y, 0)$) est appliquée pour garantir la positivité numérique. Le choix est motivé par la simplicité, la robustesse et le recours naturel à LAPACK ; pour une précision supérieure on utiliserait un schéma d'ordre plus élevé (Radau, BDF) [15].

Protocole de vérification

Nous vérifions le schéma selon trois critères, avec $y(0) = (1000,\, 0,\, 10^{-3})$ et $\eta = 0$. La solution de référence est calculée avec le solveur implicite d'ordre élevé `Radau` (SciPy, tolérances 10^{-12}).

(i) Stabilité. À $h = 0{,}1$ jour, Euler explicite diverge (valeurs non finies avant 30 jours), tandis qu'Euler implicite reste borné et reproduit la phase de pic viral. Le schéma implicite reste stable même pour $h = 0{,}5$ et $h = 1$ jour (figure 8.1b), au prix d'un décalage temporel du pic.

(ii) Convergence. Le tableau 8.1 donne l'erreur relative (norme euclidienne des erreurs relatives par composante) sur $[0, 10]$ jours et l'ordre observé $p \approx \log_2(e_h / e_{h/2})$.

Table 8.1: Erreur du schéma d'Euler implicite à $t = 10$ j par rapport à la solution de référence.

Pas h (jour)	Erreur relative	Ordre observé
$0{,}4$	$2{,}3 \times 10^{2}$	–
$0{,}2$	13	$4{,}1$
$0{,}1$	$2{,}6$	$2{,}3$
$0{,}05$	$0{,}92$	$1{,}5$
$0{,}025$	$0{,}39$	$1{,}2$
$0{,}0125$	$0{,}18$	$1{,}1$

L'ordre observé tend vers 1 lorsque h diminue, conformément à la théorie. Les erreurs absolues restent élevées aux grands pas parce que $t = 10$ j se situe en pleine croissance exponentielle du virus (V est multiplié par un facteur voisin de $e^{c \cdot \text{croissance}}$ sur quelques jours), phase où l'erreur d'un schéma d'ordre 1 est amplifiée ; cela confirme qu'un pas $h \leq 0{,}0125$ j (ou un schéma d'ordre supérieur) est nécessaire pour une précision quantitative sur la primo-infection.

(iii) Cohérence asymptotique. Un point fixe du schéma (8.1) est exactement un équilibre de f (car $f(y) = 0$ donne $\Delta y = 0$). Nous l'avons vérifié : avec $h = 0{,}1$ jour, la solution à $t = 600$ j coïncide avec $E^* = (250;\, 15;\, 652{,}17)$ à 2×10^{-5} près en erreur relative, écart dû uniquement à la décroissance lente de l'oscillation résiduelle ($e^{-0{,}0197\,t}$). Les équilibres calculés par Newton (section 7.3) et par intégration en temps long sont donc cohérents.

Mini-résumé du chapitre 8 : intégration numérique

1. Le système est raide (rapport des valeurs propres ≈ 1200) ; Euler explicite exige $h < 0{,}085$ j et diverge à $h = 0{,}1$ j.
2. Le schéma retenu est l'Euler implicite linéarisé résolu par `DGESV`, stable sans contrainte sur h.
3. Vérification : ordre observé $\to 1$, stabilité à grands pas, point fixe $=$ équilibre.
4. Précision quantitative : pas fin ou schéma d'ordre supérieur.

Chapitre 9

Résultats et interprétation médicale

9.1 Primo-infection et portrait de phase

Partant de $T = 1000$ (sujet sain), $I = 0$ et d'une trace de virus $V = 10^{-3}$ virion/μL, la simulation du modèle non traité (figure 9.1) montre :

- un **pic de virémie** d'environ $1{,}7 \times 10^4$ virions/μL atteint vers le jour 15 ;
- une **chute transitoire des CD4+** jusqu'à environ 49 cellules/μL ;
- puis des oscillations amorties (période ≈ 53 j) vers l'équilibre chronique $E^* = (250,\ 15,\ 652)$.

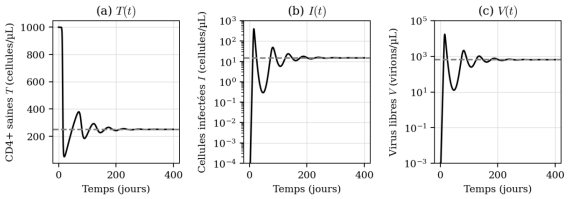


Figure 9.1: Primo-infection : $T(t)$, $I(t)$ et $V(t)$.

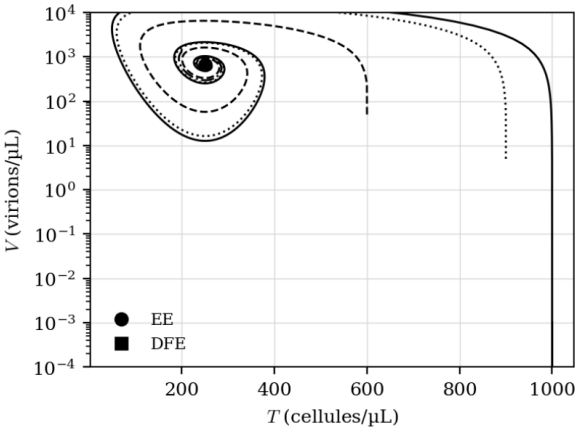


Figure 9.2: Portrait de phase (T, V) depuis plusieurs conditions initiales.

Interprétation. Le pic viral massif suivi d'un déclin est la signature de la phase aiguë : le virus consomme brutalement les cellules cibles, puis il est limité par leur raréfaction. Sur la figure 9.2, toutes les trajectoires s'enroulent en spirale vers un même point E^*, ce qui illustre la stabilité globale de l'équilibre endémique pour $\mathcal{R}_0 > 1$.

9.2 Efficacité thérapeutique et rebond viral

Seuil d'efficacité. Pour $\eta = 50\,\%$, $70\,\%$ et $95\,\%$, on obtient $\mathcal{R}_0 = 2{,}0$, $1{,}2$ et $0{,}2$. Seul le dernier cas conduit à l'élimination du virus (figure 9.3). En partant de l'équilibre chronique et avec $\eta = 95\,\%$, la charge virale passe sous le seuil d'indétectabilité de $0{,}05$ virion/μL en environ 20

jours et les CD4+ remontent (environ $900/\mu$L après 200 jours). Pour $\eta = 70\,\%$, la charge virale passe elle aussi sous ce seuil (en environ 34 jours) et descend jusqu'à $\approx 10^{-6}$ virion/μL, mais ce n'est qu'un *répit* : comme $\mathcal{R}_0 = 1{,}2 > 1$, le virus n'est pas éliminé et remonte lentement (en plusieurs centaines de jours) vers un équilibre endémique à ≈ 145 virions/μL. À $\eta = 50\,\%$, la charge virale diminue transitoirement puis *rebondit* en oscillant vers un équilibre à ≈ 435 virions/μL. Une charge virale indétectable n'est donc pas une preuve de guérison tant que $\eta < \eta_c$.

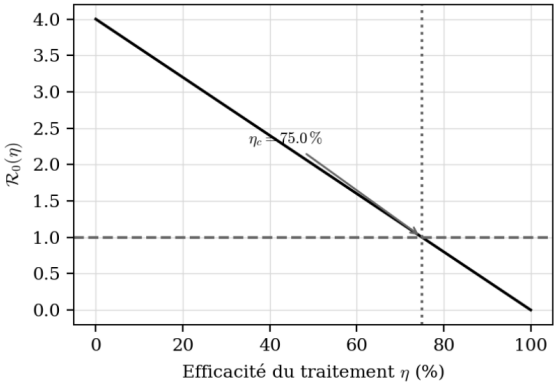


Figure 9.3: $\mathcal{R}_0$ en fonction de l'efficacité η.

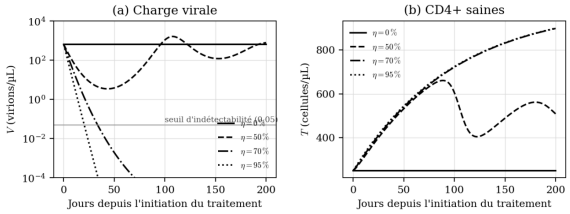


Figure 9.4: Charge virale (a) et CD4+ (b) pour plusieurs efficacités η.

Bifurcation et sensibilité. La figure 9.5 représente les équilibres T^*, I^*, V^* en fonction de $\mathcal{R}_0$: à $\mathcal{R}_0 = 1$, E_0 et E^* échangent leur stabilité (bifurcation transcritique). La figure 9.6 montre les indices de sensibilité normalisés $\Upsilon_q^{\mathcal{R}_0} = \frac{\partial \mathcal{R}_0}{\partial q}\frac{q}{\mathcal{R}_0}$:

$$\Upsilon_\beta^{\mathcal{R}_0} = \Upsilon_p^{\mathcal{R}_0} = \Upsilon_s^{\mathcal{R}_0} = +1, \qquad \Upsilon_d^{\mathcal{R}_0} = \Upsilon_\delta^{\mathcal{R}_0} = \Upsilon_c^{\mathcal{R}_0} = -1,$$

puisque $\mathcal{R}_0$ est un monôme en les paramètres. Une variation de $1\,\%$ de β, p ou s modifie $\mathcal{R}_0$ de $+1\,\%$; celle de d, δ ou c de $-1\,\%$. Pour la charge virale d'équilibre, nous trouvons $\Upsilon_s^{V^*} = \Upsilon_p^{V^*} = +4/3$, $\Upsilon_\delta^{V^*} = \Upsilon_c^{V^*} = -4/3$, $\Upsilon_\beta^{V^*} = +1/3$, $\Upsilon_d^{V^*} = -1/3$: l'infectivité β influence $\mathcal{R}_0$ mais bien moins la charge virale chronique.

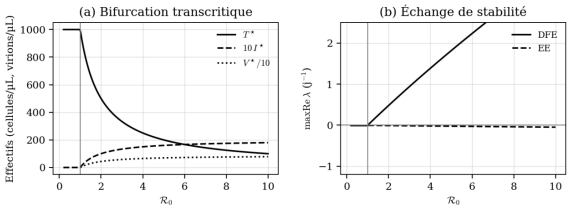


Figure 9.5: Bifurcation transcritique à $\mathcal{R}_0 = 1$ et échange de stabilité.

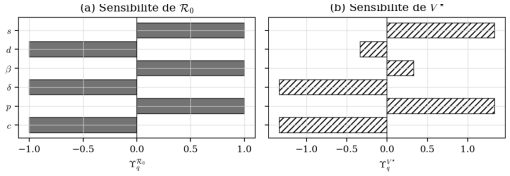


Figure 9.6: Indices de sensibilité de $\mathcal{R}_0$ et de V^*.

Danger de l'interruption du traitement. Pour un patient traité à $\eta = 95\,\%$ jusqu'au jour 100 puis *sans traitement*, le stock de CD4+ s'est reconstitué ($T \approx 724$ cellules/μL au jour 100) alors que la charge virale est indétectable. À l'arrêt, le virus résiduel dispose de ces cellules cibles abondantes : la charge virale subit un **rebond** d'un pic d'environ $1{,}2 \times 10^4$ virions/μL (jour ≈ 150), soit environ 18 **fois le niveau chronique** (652), avant de retourner vers E^* en oscillant (figure 9.7). Ce phénomène est une *amplification transitoire* : bien que E^* soit stable,

des trajectoires partant d'un état très éloigné croissent fortement avant de décroître. Il justifie l'exigence d'une observance continue du traitement.

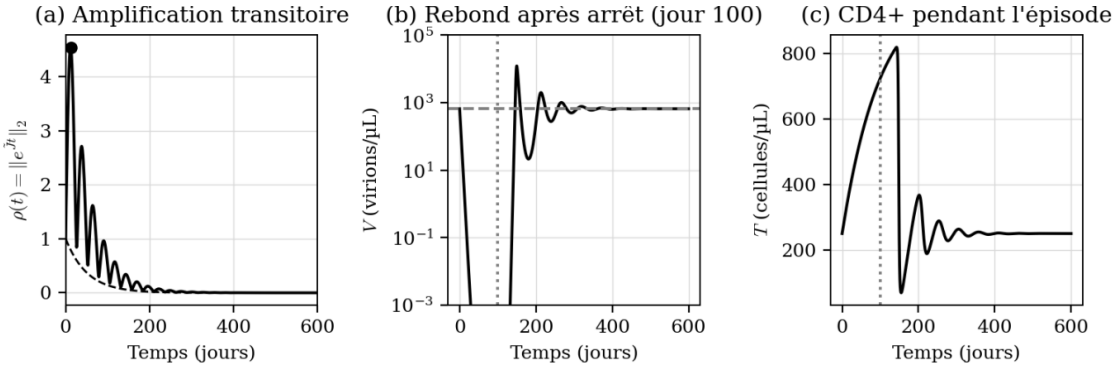


Figure 9.7: Amplification transitoire et rebond viral après arrêt du traitement.

Mini-résumé du chapitre 9 : interprétation médicale

1. La primo-infection produit un pic de virémie ($\approx 1{,}7 \times 10^4$) et un effondrement transitoire des CD4+.
2. Il faut $\eta > 75\,\%$ pour éradiquer le virus ; $\eta = 95\,\%$ rend la charge virale indétectable en ≈ 20 jours.
3. L'arrêt du traitement provoque un rebond viral d'environ 18 fois le niveau chronique.
4. $\mathcal{R}_0$ est également sensible à s, β, p ($+1$) et à d, δ, c (-1).

Chapitre 10

Modèle épidémiologique de population : S, I_1, I_2

10.1 Formulation du modèle de population

À l'échelle d'une communauté, la population $N(t)$ est divisée en trois groupes :

- $S(t)$: individus sains, susceptibles d'être infectés ;
- $I_1(t)$: personnes infectées par le VIH, contagieuses mais sans symptômes ;
- $I_2(t)$: personnes au stade SIDA.

Le modèle s'écrit

$$\begin{cases} \dfrac{dS}{dt} = \Lambda_a N - \beta S \dfrac{I_1 + \varepsilon I_2}{N} - \mu_a S \\ \dfrac{dI_1}{dt} = \beta S \dfrac{I_1 + \varepsilon I_2}{N} - \sigma I_1 - \mu_a I_1 \\ \dfrac{dI_2}{dt} = \sigma I_1 - \mu_2 I_2 - \mu_a I_2 \end{cases} \tag{10.1}$$

avec $N = S + I_1 + I_2$, Λ_a le taux de natalité, μ_a la mortalité naturelle, σ le taux de progression vers le SIDA, μ_2 la surmortalité liée au SIDA et $\varepsilon < 1$ la contagiosité relative au stade SIDA.

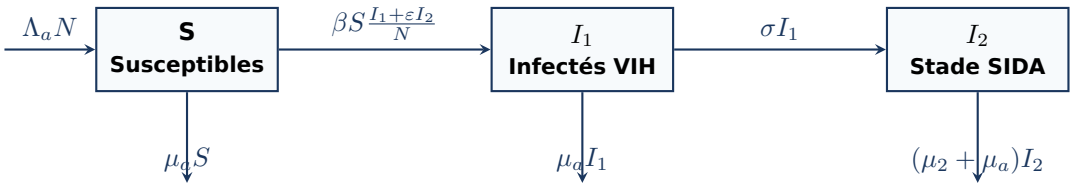


Figure 10.1: Schéma de transfert du modèle de population S–I_1–I_2.

Pourquoi diviser par N dans le terme d'infection ?

Dans les infections sexuellement transmissibles, le nombre de partenaires d'un individu ne dépend pas de la taille de la population. L'incidence est donc **standardisée** : $\beta S \frac{I_1+\varepsilon I_2}{N}$ (β est le taux de contact efficace par individu), et non proportionnelle à N (action de masse).

Passage aux proportions. Posons $s = S/N$, $i_1 = I_1/N$, $i_2 = I_2/N$ avec $s = 1 - i_1 - i_2$. Comme $N'/N = \Lambda_a - \mu_a - \mu_2\, i_2$, le système se réduit à deux équations (implémentées par `f_B` dans le code) :

$$\begin{aligned} i_1' &= \beta\, s\,(i_1 + \varepsilon i_2) - (\sigma + \Lambda_a)\, i_1 + \mu_2\, i_1 i_2, \\ i_2' &= \sigma i_1 - (\mu_2 + \Lambda_a)\, i_2 + \mu_2\, i_2^2. \end{aligned}$$

10.2 Nombre de reproduction de base et équilibres

Au point sans maladie ($s = 1$), on applique la même méthode qu'au chapitre 6 aux variables infectées (i_1, i_2) : F contient les *nouvelles infections* et V_m les *passages et sorties* (progression vers le SIDA, mortalité). On obtient

$$F = \begin{pmatrix} \beta & \beta\varepsilon \\ 0 & 0 \end{pmatrix}, \quad V_m = \begin{pmatrix} \sigma + \Lambda_a & 0 \\ -\sigma & \mu_2 + \Lambda_a \end{pmatrix} \implies \boxed{\mathcal{R}_0^{\text{pop}} = \frac{\beta}{\sigma + \Lambda_a}\Big(1 + \frac{\varepsilon\sigma}{\mu_2 + \Lambda_a}\Big)}$$

Les deux termes s'interprètent comme la contribution d'un individu I_1 pendant sa phase asymptomatique, puis pendant sa phase SIDA (affaiblie d'un facteur ε). Avec les valeurs calibrées $\Lambda_a = 0{,}04$, $\sigma = 0{,}10$, $\mu_2 = 0{,}50$, $\varepsilon = 0{,}30$ et $\beta = 0{,}2387$ (choisi pour obtenir $\mathcal{R}_0^{\text{pop}} = 1{,}8$), on a $\mathcal{R}_0^{\text{pop}} = 1{,}8$. Le seuil critique est

$$\beta_c = \frac{\sigma + \Lambda_a}{1 + \varepsilon\sigma/(\mu_2 + \Lambda_a)} \approx 0{,}133.$$

Pour $\mathcal{R}_0^{\text{pop}} > 1$, le système converge vers un équilibre endémique stable d'environ $s^* = 32{,}2\,\%$, $i_1^* = 56{,}1\,\%$, $i_2^* = 11{,}6\,\%$, soit une **prévalence** totale de $67{,}8\,\%$; la jacobienne réduite en ce point a pour valeurs propres $-0{,}098$ et $-0{,}473$ (réelles et négatives : l'approche est monotone, sans oscillation). Partant d'une faible prévalence initiale, la moitié de la valeur de plateau est atteinte vers l'année 59 et $90\,\%$ vers l'année 81.

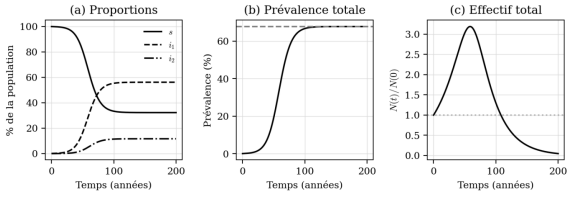


Figure 10.2: Dynamique épidémique sans intervention : proportions, prévalence et effectif total.

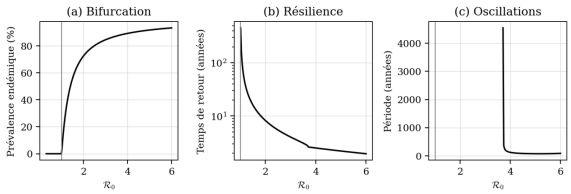


Figure 10.3: Bifurcation selon $\mathcal{R}_0^{\text{pop}}$: prévalence endémique, temps de retour, période.

Lecture des figures. La figure 10.2 montre une croissance lente puis rapide de la prévalence vers son plateau d'environ $68\,\%$ (ordre de grandeur de plusieurs décennies pour atteindre l'essentiel du plateau). La figure 10.3 confirme la bifurcation en $\mathcal{R}_0^{\text{pop}} = 1$ (soit $\beta = \beta_c$) : en dessous, l'épidémie s'éteint ; au-dessus, la prévalence endémique croît avec $\mathcal{R}_0$, et le temps de retour à l'équilibre diverge près du seuil (ralentissement critique).

10.3 Scénarios d'intervention de santé publique

On modélise trois interventions démarrant à l'an 30 :

- **Prévention** (préservatifs, éducation) : $\beta \to 0{,}5\,\beta$;

- **Traitement** (antirétroviraux précoces) : la surmortalité liée au SIDA tombe de $\mu_2 = 0{,}5$ à $0{,}1$;
- **Combinaison** des deux.

Table 10.1: Scénarios d'intervention : $\mathcal{R}_0^{\text{pop}}$ et prévalence à long terme (simulation sur 300 ans).

Scénario	$\mathcal{R}_0^{\text{pop}}$	**Prévalence à 300 ans**
Sans intervention	$1{,}80$	$67{,}8\,\%$
Prévention seule ($\beta \times 0{,}5$)	$0{,}90$	$0{,}07\,\%$
Traitement seul ($\mu_2 = 0{,}1$)	$2{,}07$	$64{,}0\,\%$
Prévention + traitement	$1{,}04$	$4{,}6\,\%$

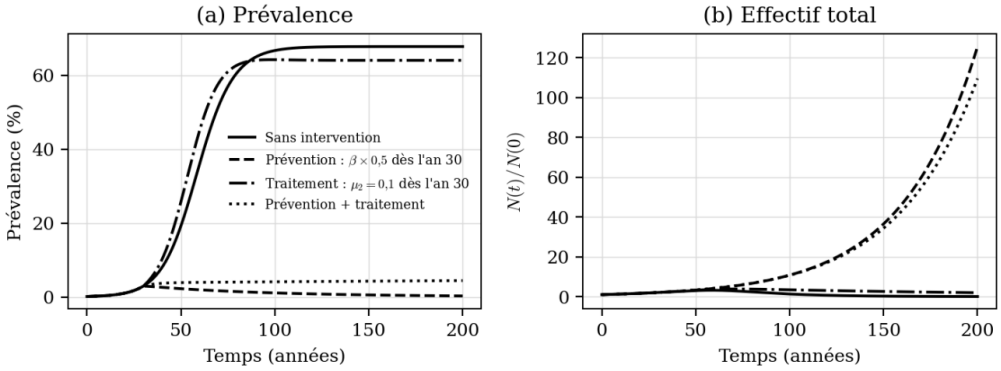


Figure 10.4: Comparaison de scénarios de santé publique (prévalence et effectif total).

Interprétation. Les résultats du tableau 10.1 appellent trois remarques.

1. Le traitement seul *augmente* $\mathcal{R}_0^{\text{pop}}$ (de $1{,}80$ à $2{,}07$) dans ce modèle, car il prolonge la vie des individus infectés, donc leur période de contagiosité : il sauve des vies mais ne bloque pas la transmission et la prévalence reste élevée ($64\,\%$).
2. La prévention seule, en divisant β par deux, fait passer $\mathcal{R}_0^{\text{pop}}$ sous 1 : l'épidémie s'éteint.
3. La combinaison est ici *moins* efficace que la prévention seule pour l'extinction, car le traitement allonge les durées d'infection ; avec $\mathcal{R}_0^{\text{pop}} = 1{,}04$ juste au-dessus du seuil, la prévalence retombe à un niveau bas mais l'épidémie n'est pas éteinte et peut se maintenir longtemps.

Il en ressort que, dans ce modèle, la prévention est le levier décisif, le traitement ayant surtout un bénéfice individuel (survie) et nécessitant d'être associé à des mesures de réduction de la transmission. Ce résultat dépend de l'hypothèse que le traitement n'affecte pas β ; en réalité, un traitement efficace réduit aussi la contagiosité (voir les limites du modèle).

Mini-résumé du chapitre 10 : modèle de population

1. Trois groupes : S, I_1 (asymptomatiques), I_2 (SIDA), avec incidence standardisée.
2. $\mathcal{R}_0^{\text{pop}} = \frac{\beta}{\sigma+\Lambda_a}\left(1 + \frac{\varepsilon\sigma}{\mu_2+\Lambda_a}\right) = 1{,}8$; $\beta_c \approx 0{,}133$; prévalence endémique $\approx 68\,\%$.
3. Prévention : décisive ($\mathcal{R}_0 = 0{,}9$) ; traitement seul : prolonge la vie mais pas de contrôle de la transmission ($\mathcal{R}_0 = 2{,}07$).

Chapitre 11

Benchmark haute performance BLAS / LAPACK

11.1 Protocole de mesure

Le script `benchmark_blas.py` (annexe B) mesure :

- le produit matriciel `DGEMM` (BLAS niveau 3), de coût $2n^3$ opérations flottantes ;
- la résolution de système `DGESV` (LAPACK), de coût $\frac{2}{3}n^3$;
- des implémentations **Python naïf** (boucles et listes, fonctions `matmul_naive` et `gauss_naive`) servant de référence de comparaison.

Chaque mesure retient le **meilleur temps de plusieurs répétitions** (fonction `best_time`) pour limiter l'effet du bruit système. Le débit est GFLOPS $=$ opérations$/(t \times 10^9)$. Les matrices sont tirées avec un générateur aléatoire de graine fixée (`default_rng(2026)`) et stockées en ordre Fortran. Le backend actif est identifié par `threadpoolctl` (section 4.1).

Conditions de mesure du tableau et de la figure

Les tableaux 11.1 et 11.2 ont été obtenus avec `benchmark_blas.py` dans l'environnement suivant : processeur Intel Xeon à $2{,}8$ GHz (2 cœurs), Python 3.13, NumPy 2.5.3, SciPy 1.18.1, backend **OpenBLAS** 0.3.34. La figure 11.1 présente la même mesure effectuée sur la machine du groupe. Les valeurs absolues dépendent du matériel ; ce sont les ordres de grandeur et les tendances qui comptent.

11.2 Résultats et analyse du gain de performance

Gain par rapport à Python naïf. Le tableau 11.1 compare, à même taille n, le code Python naïf et les routines BLAS/LAPACK.

Table 11.1: Temps d'exécution et facteur d'accélération (BLAS/LAPACK contre Python naïf).

	Produit matriciel			**Résolution** $Ax = b$		
n	Naïf (ms)	`DGEMM` (ms)	Gain	Naïf (ms)	`DGESV` (ms)	Gain
50	6,3	0,009	×720	2,6	0,026	×100
100	45,8	0,048	×950	19,6	0,094	×210
200	380,8	0,39	×980	142,5	0,45	×310

Le gain est de l'ordre de 10^3 **pour le produit matriciel** et de **plusieurs centaines pour la résolution de système**, dès $n = 200$. Il croît avec n pour `DGESV` : le surcoût fixe de l'interpréteur Python (boucles, objets) pèse davantage lorsque le nombre d'opérations augmente, alors que les noyaux BLAS travaillent par blocs vectorisés. Le code naïf étant de complexité $O(n^3)$ avec une constante très élevée, il devient impraticable bien avant $n = 1000$ (plusieurs minutes), ce qui n'est donc pas mesuré ici.

Passage à l'échelle. Le tableau 11.2 donne les temps et débits de `DGEMM` et `DGESV` pour de grandes matrices (figure 11.1).

Table 11.2: Temps et débit de `DGEMM` et `DGESV` pour de grandes tailles (OpenBLAS, 2 cœurs).

n	`DGEMM` (ms)	GFLOPS	`DGESV` (ms)	GFLOPS
100	0,04	50,5	0,09	7,0
200	0,33	49,2	0,44	12,1
400	2,57	49,8	2,76	15,5
800	19,4	52,8	18,7	18,3
1600	135,6	60,4	89,9	30,4

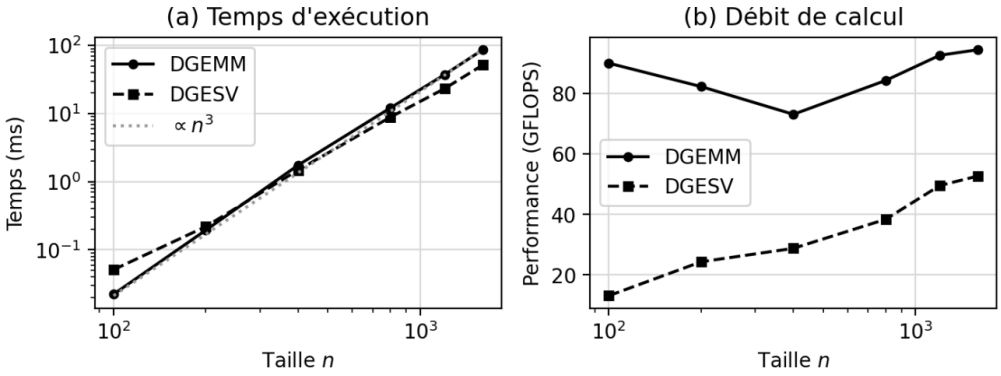


Figure 11.1: Temps d'exécution (a) et débit en GFLOPS (b) de `DGEMM` et `DGESV` en fonction de n (machine du groupe).

Analyse.

- **Loi en n^3.** Les temps croissent comme n^3 (droite de pente 3 en échelle logarithmique), conformément aux coûts théoriques : multiplier n par 2 multiplie le temps par environ 8.
- **`DGEMM` près de son régime stable.** Son débit varie peu avec n : le niveau 3 est limité par le calcul, non par la mémoire (chapitre 2).
- **`DGESV` moins efficace que `DGEMM`, mais croissant.** Environ $\frac{2}{3}n^3$ opérations dont une partie (la factorisation des panneaux) se fait au niveau 2, donc limitée par la mémoire ; la part de niveau 3 grandit avec n, d'où un débit croissant.
- **Rôle de la bibliothèque.** MKL et OpenBLAS appliquent les mêmes principes (blocage, vectorisation, multithreading) ; les écarts entre eux dépendent du processeur. L'exécution d'une comparaison directe OpenBLAS/MKL demande un environnement `conda` avec MKL ; elle n'a pas été réalisée ici.

Mini-résumé du chapitre 11 : benchmark

1. BLAS/LAPACK sont environ 10^3 fois plus rapides que le Python naïf pour le produit matriciel, et plusieurs centaines de fois pour la résolution de système, dès $n = 200$.
2. Les temps suivent la loi n^3 ; `DGEMM` atteint un débit presque constant, `DGESV` un débit croissant avec n.
3. Les valeurs absolues dépendent du matériel et du backend (OpenBLAS ou MKL).

Conclusion et perspectives

Ce projet a mis en œuvre la chaîne complète qui mène d'un modèle biologique à une simulation numérique fiable :

1. **Outils.** MKL, BLAS et LAPACK forment une pile cohérente : BLAS standardise les noyaux, LAPACK construit les solveurs par blocs, MKL ou OpenBLAS les optimise pour le matériel.
2. **Modèle intra-hôte.** Il est borné et positif ; $\mathcal{R}_0(\eta) = \frac{(1-\eta)\beta p s}{d\delta c}$ vaut 4 sans traitement et fixe un seuil d'efficacité $\eta_c = 75\,\%$. L'équilibre sain est globalement stable si $\mathcal{R}_0 \leq 1$; l'équilibre endémique est stable sinon, avec des oscillations de période ≈ 53 jours.
3. **Numérique.** L'équilibrage `DGEEQU` ramène le conditionnement de la jacobienne de 10^5 à quelques dizaines ; Newton + `DGESV` converge quadratiquement ; l'Euler implicite linéarisé surmonte la raideur et converge à l'ordre 1.
4. **Résultats médicaux.** Pic de virémie en primo-infection, élimination du virus pour $\eta > 75\,\%$ et rebond d'environ 18 fois le niveau chronique après arrêt du traitement.
5. **Population.** $\mathcal{R}_0^{\text{pop}} = 1{,}8$; la prévention est le levier décisif, le traitement seul ne bloquant pas la transmission dans ce modèle.
6. **Performance.** Gain de l'ordre de 10^3 pour `DGEMM` et de plusieurs centaines pour `DGESV` par rapport à Python naïf.

Perspectives. (i) Utiliser un intégrateur raide d'ordre supérieur (BDF, Radau) ; (ii) mesurer explicitement MKL contre OpenBLAS sur une même machine, avec variation du nombre de threads ; (iii) étendre à des modèles spatiaux (équations aux dérivées partielles) en exploitant les solveurs LAPACK pour matrices bandes ; (iv) comparer des exécutions sur GPU (cuBLAS).

Limites du modèle

- **Modèle intra-hôte :** il ignore la réponse immunitaire adaptative, les cellules latentes (réservoir), les mutations et résistances, et suppose une efficacité η constante, une incidence bilinéaire et des paramètres identiques pour tous les patients. L'équilibre chronique ($V^* \approx 650$ virions/μL) et les oscillations amorties qui l'entourent sont des propriétés de ce modèle simple, pas nécessairement des observations cliniques.
- **Modèle de population :** mélange homogène, pas de structure d'âge, de sexe ou de réseaux de partenaires ; un traitement n'y agit que sur la mortalité μ_2, alors qu'il réduit aussi la contagiosité (ce qui modifierait la conclusion sur la combinaison) ; β est *calibré* pour imposer $\mathcal{R}_0^{\text{pop}} = 1{,}8$, et non estimé sur des données réelles ; la natalité Λ_a et la mortalité μ_a différentes impliquent un effectif total qui varie fortement sur de longues durées.

- **Numérique :** l'Euler implicite est d'ordre 1 (précision limitée aux grands pas) ; les paramètres du tableau 6.1 sont des valeurs de littérature arrondies.
- **Benchmark :** résultats obtenus pour un matériel et un backend donnés ; la comparaison OpenBLAS/MKL reste à réaliser.

Bibliographie

[1] M. A. Nowak, R. M. May, *Virus Dynamics: Mathematical Principles of Immunology and Virology*, Oxford University Press, 2000.

[2] A. S. Perelson, P. W. Nelson, *Mathematical analysis of HIV-1 dynamics in vivo*, SIAM Review 41(1), 1999, 3–44.

[3] A. S. Perelson et al., *HIV-1 dynamics in vivo: virion clearance rate, infected cell life-span, and viral generation time*, Science 271, 1996, 1582–1586.

[4] O. Diekmann, J. A. P. Heesterbeek, J. A. J. Metz, *On the definition and the computation of the basic reproduction ratio R_0 in models for infectious diseases in heterogeneous populations*, J. Math. Biol. 28, 1990, 365–382.

[5] P. van den Driessche, J. Watmough, *Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission*, Math. Biosci. 180, 2002, 29–48.

[6] A. Korobeinikov, *Global properties of basic virus dynamics models*, Bull. Math. Biol. 66, 2004, 879–883.

[7] P. De Leenheer, H. L. Smith, *Virus dynamics: a global analysis*, SIAM J. Appl. Math. 63(4), 2003, 1313–1327.

[8] R. M. Anderson, R. M. May, *Infectious Diseases of Humans: Dynamics and Control*, Oxford University Press, 1991.

[9] E. Anderson et al., *LAPACK Users' Guide*, 3e éd., SIAM, 1999.

[10] C. L. Lawson, R. J. Hanson, D. R. Kincaid, F. T. Krogh, *Basic linear algebra subprograms for Fortran usage*, ACM TOMS 5(3), 1979, 308–323.

[11] J. J. Dongarra, J. Du Croz, S. Hammarling, I. S. Duff, *A set of level 3 basic linear algebra subprograms*, ACM TOMS 16(1), 1990, 1–17.

[12] K. Goto, R. A. van de Geijn, *Anatomy of high-performance matrix multiplication*, ACM TOMS 34(3), 2008, art. 12.

[13] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4e éd., Johns Hopkins University Press, 2013.

[14] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2e éd., SIAM, 2002.

[15] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, Springer, 1996.

Annexe A

Code source du mini-projet (`vih_lapack.py`)

A.1 Le module du modèle : `vih_lapack.py`

Ce module contient les outils LAPACK (`eigvals_lapack`, `lu_solve`, `cond1`, ...), le modèle intra-hôte (`rhs_h`, `jac_h`, `ee_h`, `newton_h`) et le modèle de population (`f_B`, `J_B`).

```python
# -*- coding: utf-8 -*-
"""
Mini-projet OCS -- Groupe 5 -- Modelisation du VIH-SIDA avec LAPACK (Python)
============================================================================
Partie A : modele intra-hote (T, I, V) avec traitement antiretroviral d'efficacite eta
Partie B : modele de population (S, I1, I2) avec mortalites naturelle et liee au SIDA

Routines LAPACK (via scipy.linalg.lapack) :
  dgetrf / dgetrs / dgesv : factorisation LU et resolution de systemes lineaires
  dgecon                  : estimation du conditionnement (norme 1)
  dgeev                   : valeurs propres et vecteurs propres (matrice generale)
  dgesdd                  : decomposition en valeurs singulieres (norme spectrale)
  dsyevd                  : valeurs propres d'une matrice symetrique (reactivite)
"""

import os, json, time
import numpy as np
from scipy.linalg import lapack, expm, hilbert
from scipy.integrate import solve_ivp
from scipy.optimize import brentq
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# ---------------------------------------------------------------------------
# Outils LAPACK
# ---------------------------------------------------------------------------
def eigvals_lapack(A):
    wr, wi, _, _, info = lapack.dgeev(np.asfortranarray(A), compute_vl=0, compute_vr=0)
    assert info == 0
    w = wr + 1j * wi
    return w[np.argsort(-w.real)]

def lu_solve(A, b):
    """Resolution par dgetrf + dgetrs ; retourne x, rcond (dgecon), pivots."""
    A = np.asfortranarray(A, dtype=float)
    lu, piv, info = lapack.dgetrf(A)
    assert info == 0
    x, info = lapack.dgetrs(lu, piv, b)
```

```
    rcond, _ = lapack.dgecon(lu, np.linalg.norm(A, 1), norm="1")
    return x, rcond, piv

def cond1(A):
    A = np.asfortranarray(A, dtype=float)
    lu, piv, info = lapack.dgetrf(A)
    rcond, _ = lapack.dgecon(lu, np.linalg.norm(A, 1), norm="1")
    return 1.0 / rcond

def sigma_max(Mx):
    u, s, vt, info = lapack.dgesdd(np.asfortranarray(Mx), compute_uv=0)
    return s[0]

def sym_eigmax(Mx):
    return lapack.dsyevd(np.asfortranarray(Mx), compute_v=0)[0].max()

# ============================================================================
# PARTIE A : MODELE INTRA-HOTE
# ============================================================================
PA = dict(s=10.0,      # production de CD4+ (cellules / L / jour)
          d=0.01,      # mortalite naturelle des CD4+ (jour^-1) => T0 = s/d = 1000
          delta=0.5,   # mortalite des cellules infectees (jour^-1)
          p=1000.0,    # production virale (virions / cellule infectee / jour)
          c=23.0,      # clairance virale (jour^-1)
          beta=4.6e-5, # infectivite (L / virion / jour)
          eta=0.0)     # efficacite du traitement antiretroviral

def R0_h(P, eta=None):
    e = P["eta"] if eta is None else eta
    return (1 - e) * P["beta"] * P["p"] * (P["s"] / P["d"]) / (P["delta"] * P["c"])

def rhs_h(t, y, P, eta=None):
    T, I, V = y
    e = P["eta"] if eta is None else eta
    inf = (1 - e) * P["beta"] * T * V
    return np.array([P["s"] - P["d"] * T - inf, inf - P["delta"] * I, P["p"] * I - P["c"] * V])

def jac_h(y, P, eta=None):
    T, I, V = y
    e = P["eta"] if eta is None else eta
    b = (1 - e) * P["beta"]
    return np.array([[-P["d"] - b * V, 0.0, -b * T],
                     [b * V, -P["delta"], b * T],
                     [0.0, P["p"], -P["c"]]])

def dfe_h(P):
    return np.array([P["s"] / P["d"], 0.0, 0.0])

def ee_h(P, eta=None):
    e = P["eta"] if eta is None else eta
    R0 = R0_h(P, e)
    T0 = P["s"] / P["d"]
    Ts = T0 / R0
    Is = (P["s"] - P["d"] * Ts) / P["delta"]
    Vs = P["p"] * Is / P["c"]
    return np.array([Ts, Is, Vs])

def newton_h(y0, P, eta=None, tol=1e-12, maxit=50):
    """Newton : J(y) dy = -f(y), resolu par dgesv."""
    y = np.array(y0, float)
    scale = np.array([P["s"] / P["d"], 1.0, 1.0])
    for k in range(maxit):
        f = rhs_h(0, y, P, eta)
```

```
        lu, piv, dy, info = lapack.dgesv(np.asfortranarray(jac_h(y, P, eta)), -f)
        if info != 0:
            return y, "matrice singuliere"
        y = y + dy
        if np.linalg.norm(dy / scale) < tol:
            return y, "converge"
    return y, "maxit atteint"

# ============================================================================
# PARTIE B : MODELE DE POPULATION (S, I1, I2)
# ============================================================================
PB = dict(Lam=0.04, mu=0.015, sigma=0.10, mu2=0.50, eps=0.30)

def R0_B(P, beta):
    return beta / (P["sigma"] + P["Lam"]) * (1 + P["eps"] * P["sigma"] / (P["mu2"] + P["Lam"]))

R0B = 1.8
betaB = R0B / R0_B(PB, 1.0)
PB["beta"] = betaB

def f_B(y, P):
    i1, i2 = y
    s = 1 - i1 - i2
    F = i1 + P["eps"] * i2
    return np.array([P["beta"] * s * F - (P["sigma"] + P["Lam"]) * i1 + P["mu2"] * i1 * i2,
                     P["sigma"] * i1 - (P["mu2"] + P["Lam"]) * i2 + P["mu2"] * i2 ** 2])

def J_B(y, P):
    i1, i2 = y
    s = 1 - i1 - i2
    F = i1 + P["eps"] * i2
    return np.array([[-P["beta"] * F + P["beta"] * s - (P["sigma"] + P["Lam"]) + P["mu2"] * i2,
                      -P["beta"] * F + P["beta"] * P["eps"] * s + P["mu2"] * i1],
                     [P["sigma"], -(P["mu2"] + P["Lam"]) + 2 * P["mu2"] * i2]])

if __name__ == '__main__':
    print("Modele VIH-SIDA avec LAPACK initialise avec succes.")
```

A.2 Le script de simulation : `simulation_intra.py`

Ce script utilise le module précédent. Il implémente l'Euler implicite résolu par `DGESV`, l'Euler explicite, la solution de référence (solveur `Radau`) et reproduit les résultats numériques des chapitres 7, 8, 9 et 10 (conditionnement et équilibrage, ordre de convergence, primo-infection, traitement, rebond, scénarios de population).

```
# -*- coding: utf-8 -*-
"""
Simulation du modele intra-hote (T, I, V) avec LAPACK -- Groupe 5
================================================================
Ce script reproduit les resultats numeriques des chapitres 7, 8 et 9 :
  1. conditionnement de la jacobienne et equilibrage par DGEEQU ;
  2. schema d'Euler implicite lineaire resolu par DGESV (+ Euler explicite) ;
  3. verification : stabilite, ordre de convergence, point fixe ;
  4. primo-infection, efficacite therapeutique, rebond apres arret du traitement ;
  5. scenarios d'intervention du modele de population (S, I1, I2).

Usage : python simulation_intra.py   (vih_lapack.py doit etre dans le meme dossier)
"""

import numpy as np
from scipy.linalg import lapack
```

```python
from scipy.integrate import solve_ivp

import vih_lapack as v

P = dict(v.PA)                        # parametres du modele intra-hote
Y0 = np.array([1000.0, 0.0, 1e-3])    # sujet sain + trace de virus


# ---------------------------------------------------------------------------
# Schemas d'integration
# ---------------------------------------------------------------------------
def euler_implicite(y0, h, t_final, eta_fn=lambda t: P["eta"]):
    """Euler implicite lineaire : (I - h J) dy = h f(y), resolu par DGESV."""
    n = int(round(t_final / h))
    y = np.array(y0, float)
    traj = [y.copy()]
    for k in range(n):
        eta = eta_fn(k * h)
        f = v.rhs_h(0.0, y, P, eta)
        J = v.jac_h(y, P, eta)
        M = np.asfortranarray(np.eye(3) - h * J)
        _, _, dy, info = lapack.dgesv(M, h * f)
        assert info == 0, "DGESV a echoue"
        y = np.maximum(y + dy, 0.0)          # positivite numerique
        traj.append(y.copy())
    return np.array(traj)


def euler_explicite(y0, h, t_final):
    n = int(round(t_final / h))
    y = np.array(y0, float)
    with np.errstate(all="ignore"):
        for _ in range(n):
            y = y + h * v.rhs_h(0.0, y, P)
    return y


def reference(y0, t_final, eta_fn=lambda t: P["eta"]):
    """Solution de reference : solveur implicite d'ordre eleve (Radau)."""
    return solve_ivp(lambda t, y: v.rhs_h(t, y, P, eta_fn(t)), [0, t_final], y0,
                     method="Radau", rtol=1e-12, atol=1e-14, dense_output=True)


# ---------------------------------------------------------------------------
# Programme principal
# ---------------------------------------------------------------------------
if __name__ == "__main__":
    ee = v.ee_h(P)
    print("R0(0) =", v.R0_h(P), "| eta_c =", 1 - 1 / v.R0_h(P))
    print("Equilibre endemique E* =", ee)
    print("Valeurs propres en E* (DGEEV) :", v.eigvals_lapack(v.jac_h(ee, P)))

    # 1. Conditionnement et equilibrage (DGEEQU)
    print("\n--- Equilibrage de la jacobienne ---")
    for nom, y in [("E*", ee), ("E0", v.dfe_h(P)), ("pic", np.array([60.0, 300.0, 17000.0]))]:
        J = v.jac_h(y, P)
        r, c, *_ = lapack.dgeequ(np.asfortranarray(J))
        print(f"{nom:4s} kappa1(J) = {v.cond1(J):.3g} | apres equilibrage = "
              f"{v.cond1(np.diag(r) @ J @ np.diag(c)):.3g}")

    # 2-3. Verification du schema
    print("\n--- Verification : ordre de convergence a t = 10 j ---")
    ref10 = reference(Y0, 10.0).y[:, -1]
```

```
    erreurs = []
    for h in [0.4, 0.2, 0.1, 0.05, 0.025, 0.0125]:
        y = euler_implicite(Y0, h, 10.0)[-1]
        erreurs.append(np.linalg.norm((y - ref10) / np.maximum(np.abs(ref10), 1e-12)))
    for i, (h, e) in enumerate(zip([0.4, 0.2, 0.1, 0.05, 0.025, 0.0125], erreurs)):
        ordre = "" if i == 0 else f"{np.log2(erreurs[i - 1] / e):.2f}"
        print(f"h = {h:7.4f}  erreur = {e:9.3g}  ordre = {ordre}")

    print("\n--- Stabilite : Euler explicite a h = 0.1 j sur 30 j ---")
    print("explicite :", euler_explicite(Y0, 0.1, 30.0), "(valeurs non finies = divergence)")
    print("implicite :", euler_implicite(Y0, 0.1, 30.0)[-1])

    print("\n--- Point fixe : implicite h = 0.1 j a t = 600 j ---")
    y600 = euler_implicite(Y0, 0.1, 600.0)[-1]
    print("erreur relative a E* :", np.abs(y600 - ee) / ee)

    # 4. Primo-infection
    print("\n--- Primo-infection (Radau) ---")
    sol = reference(Y0, 400.0)
    tt = np.linspace(0, 400, 40001)
    Y = sol.sol(tt)
    i = Y[2].argmax()
    print(f"pic de V = {Y[2, i]:.4g} virions/uL au jour {tt[i]:.1f} ; T minimal = {Y[0].min():.1f}
        ")

    # Efficacite therapeutique, a partir de l'equilibre chronique
    print("\n--- Traitement a partir de E* ---")
    for eta in (0.50, 0.70, 0.95):
        s = solve_ivp(lambda t, y: v.rhs_h(t, y, P, eta), [0, 1500], ee, method="Radau",
                      rtol=1e-10, atol=1e-20, dense_output=True)
        t2 = np.linspace(0, 1500, 150001)
        V = s.sol(t2)[2]
        indet = t2[np.argmax(V < 0.05)] if (V < 0.05).any() else None
        print(f"eta = {eta:.2f} | R0 = {v.R0_h(P, eta):.2f} | indetectable (V<0.05) au jour {indet
            } "
              f"| V(1500) = {V[-1]:.4g}")

    # Rebond apres arret du traitement (eta = 0.95 jusqu'au jour 100)
    print("\n--- Arret du traitement au jour 100 ---")
    eta_fn = lambda t: 0.95 if t < 100 else 0.0
    s1 = solve_ivp(lambda t, y: v.rhs_h(t, y, P, eta_fn(t)), [0, 100], ee, method="Radau",
                   rtol=1e-10, atol=1e-14)
    y100 = s1.y[:, -1]
    s2 = solve_ivp(lambda t, y: v.rhs_h(t, y, P, 0.0), [100, 500], y100, method="Radau",
                   rtol=1e-10, atol=1e-14, dense_output=True)
    t3 = np.linspace(100, 500, 40001)
    V3 = s2.sol(t3)[2]
    j = V3.argmax()
    print(f"T(100) = {y100[0]:.0f} ; pic de rebond V = {V3[j]:.4g} au jour {t3[j]:.1f} "
          f"(= {V3[j] / ee[2]:.1f} fois V*)")

    # 5. Modele de population : scenarios (a partir de l'an 30)
    print("\n--- Modele de population : scenarios ---")
    P0 = dict(v.PB)
    for nom, bf, m2 in [("sans intervention", 1.0, 0.5), ("prevention (beta x 0.5)", 0.5, 0.5),
                        ("traitement (mu2 = 0.1)", 1.0, 0.1), ("prevention + traitement", 0.5,
                            0.1)]:
        def f(t, y, bf=bf, m2=m2):
            Q = dict(P0)
            if t >= 30:
                Q["beta"] = P0["beta"] * bf
                Q["mu2"] = m2
            return v.f_B(y, Q)
```

```
        s = solve_ivp(f, [0, 300], [1e-3, 0.0], rtol=1e-9, atol=1e-12, max_step=0.5)
        Q = dict(P0)
        Q["beta"] = P0["beta"] * bf
        Q["mu2"] = m2
        print(f"{nom:26s} R0 = {v.R0_B(Q, Q['beta']):.3f} | prevalence a 300 ans = "
              f"{100 * s.y[:, -1].sum():.2f} %")
```

Annexe B

Code source du benchmark (`benchmark_blas.py`)

```python
# -*- coding: utf-8 -*-
"""
Benchmark BLAS / LAPACK : DGEMM (niveau 3 BLAS) et DGESV (LAPACK, LU) sur de grandes matrices.
Usage : python benchmark_blas.py
"""

import os, sys, time, json, platform
import numpy as np
from scipy.linalg import lapack, blas
from threadpoolctl import threadpool_info, threadpool_limits

rng = np.random.default_rng(2026)

def best_time(fn, rep=3):
    ts = []
    for _ in range(rep):
        t0 = time.perf_counter()
        fn()
        ts.append(time.perf_counter() - t0)
    return min(ts)

def matmul_naive(A, B):
    n = len(A)
    C = [[0.0] * n for _ in range(n)]
    Bt = list(zip(*B))
    for i in range(n):
        Ai = A[i]
        for j in range(n):
            C[i][j] = sum(a * b for a, b in zip(Ai, Bt[j]))
    return C

def gauss_naive(A, b):
    n = len(A)
    A = [row[:] + [b[i]] for i, row in enumerate(A)]
    for k in range(n):
        p = max(range(k, n), key=lambda i: abs(A[i][k]))
        A[k], A[p] = A[p], A[k]
        for i in range(k + 1, n):
            f = A[i][k] / A[k][k]
            for j in range(k, n + 1):
                A[i][j] -= f * A[k][j]
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (A[i][n] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
```

```
    return x

if __name__ == '__main__':
    info = threadpool_info()
    backend = ", ".join(sorted({f"{d.get('internal_api')} ({d.get('version')})" for d in info if d
        .get('user_api') == 'blas'}))
    ncpu = os.cpu_count()
    print("Bibliotheque BLAS/LAPACK active :", backend, "| coeurs logiques :", ncpu)

    sizes = [100, 200, 400, 800, 1600]
    for n in sizes:
        A = np.asfortranarray(rng.standard_normal((n, n)))
        B = np.asfortranarray(rng.standard_normal((n, n)))
        b = rng.standard_normal(n)

        t_gemm = best_time(lambda: blas.dgemm(1.0, A, B))
        gflops_gemm = 2 * n**3 / t_gemm / 1e9

        t_gesv = best_time(lambda: lapack.dgesv(A.copy(order="F"), b))
        gflops_gesv = (2/3) * n**3 / t_gesv / 1e9

        print(f"n={n:5d} | DGEMM: {t_gemm*1e3:8.2f} ms ({gflops_gemm:6.1f} GFLOPS) | DGESV: {
            t_gesv*1e3:8.2f} ms ({gflops_gesv:6.1f} GFLOPS)")
```

Annexe C

Guide d'exécution et reproductibilité

Ce guide permet à toute personne de refaire, pas à pas, les calculs du rapport. Les commandes sont données pour Linux/macOS (terminal) ; pour Windows, remplacer `python3` par `python` et utiliser l'invite de commandes ou PowerShell.

C.1 Contenu du dossier

```
Rapport_VIH_Groupe5/
|-- main.tex                # source LaTeX du rapport
|-- figures/                # figures (png) et logos
|-- vih_lapack.py           # modele + outils LAPACK (annexe A.1)
|-- simulation_intra.py     # simulations et verifications (annexe A.2)
|-- benchmark_blas.py       # benchmark BLAS/LAPACK (annexe B)
```

C.2 Étape 1 : préparer l'environnement Python

Il faut Python 3.9 ou plus récent. On crée un environnement isolé pour ne pas perturber le reste du système, puis on installe les bibliothèques :

```
cd Rapport_VIH_Groupe5
python3 -m venv env
source env/bin/activate            # Windows : env\Scripts\activate
pip install numpy scipy matplotlib threadpoolctl
```

Les quatre bibliothèques servent à : `numpy` (tableaux), `scipy` (accès à BLAS/LAPACK et solveur de référence), `matplotlib` (tracés, importé par le module), `threadpoolctl` (identification du backend et du nombre de threads).

C.3 Étape 2 : vérifier la bibliothèque BLAS/LAPACK active

```
python3 -c "from threadpoolctl import threadpool_info; \
[print(d['internal_api'], d['version'], d['num_threads']) \
 for d in threadpool_info()]"
```

Le programme affiche le nom du backend (`openblas` pour une installation par `pip`, `mkl` si l'environnement est configuré avec MKL), sa version et son nombre de threads. Notez ces informations : les temps du benchmark en dépendent.

C.4 Étape 3 : tester le module du modèle

```
python3 vih_lapack.py
```

Résultat attendu : `Modele VIH-SIDA avec LAPACK initialise avec succes.` Si une erreur d'importation apparaît, vérifier que l'environnement est activé et que l'étape 1 s'est bien terminée.

C.5 Étape 4 : reproduire les résultats numériques

```
python3 simulation_intra.py
```

Le script s'exécute en quelques secondes et affiche des blocs de résultats. Le tableau C.1 indique les valeurs que l'on doit retrouver (à l'arrondi près), avec la section du rapport correspondante.

Table C.1: Résultats attendus de `simulation_intra.py`.

Bloc affiché	**Valeurs attendues**	**Section**
$\mathcal{R}_0$ et seuil	$\mathcal{R}_0(0) = 4{,}0$; $\eta_c = 0{,}75$	6.2
Équilibre endémique	$E^* = (250;\ 15;\ 652{,}17)$	6.3
Valeurs propres en E^*	$-23{,}50$; $-0{,}0197 \pm 0{,}1196\,i$	6.3
Équilibrage `DGEEQU`	$\kappa_1(J(E^*)) = 1{,}5 \times 10^5 \to \approx 69$	7.2
Ordre de convergence	ordre observé $\to 1$ (de $4{,}1$ à $1{,}1$) ; erreur $0{,}18$ pour $h = 0{,}0125$	8.2
Stabilité à $h = 0{,}1$ j	explicite : nan (divergence) ; implicite : fini	8.2
Point fixe à $t = 600$ j	erreur relative $\approx 10^{-5}$	8.2
Primo-infection	pic $V \approx 1{,}7 \times 10^4$ au jour $15{,}4$; $T_{\min} \approx 49$	9.1
Traitement	$\eta = 0{,}95$: indétectable au jour ≈ 20 ; $\eta = 0{,}70$: jour ≈ 34 puis remontée ($V \approx 145$)	9.2
Arrêt du traitement	$T(100) \approx 724$; rebond $\approx 1{,}2 \times 10^4$, soit $18{,}5 \times V^*$	9.2
Scénarios (population)	$\mathcal{R}_0$ = 1,80 / 0,90 / 2,07 / 1,04 ; prévalence 67,8 / 0,07 / 64,0 / 4,6 %	10.3

Des avertissements de dépassement de capacité (`RuntimeWarning: overflow`) apparaissent lors du test d'Euler explicite : c'est normal, ils illustrent la divergence décrite au chapitre 8.

C.6 Étape 5 : refaire le benchmark

```
python3 benchmark_blas.py
```

Le script affiche d'abord le backend détecté, puis, pour $n = 100, 200, 400, 800, 1600$, le temps et le débit (GFLOPS) de `DGEMM` et `DGESV`, par exemple :

```
n=  100 | DGEMM:     0.04 ms ( 50.5 GFLOPS) | DGESV:     0.09 ms (  7.0 GFLOPS)
```

Les valeurs absolues dépendent de la machine : **ce sont les tendances qui doivent être reproduites** (temps en n^3, `DGEMM` plus rapide en GFLOPS que `DGESV`, débit croissant avec n). Pour fixer le nombre de threads :

```
OMP_NUM_THREADS=1 python3 benchmark_blas.py     # Linux / macOS
set OMP_NUM_THREADS=1 && python benchmark_blas.py   # Windows
```

Pour mesurer la scalabilité, répéter avec 1, 2, 4 threads et comparer les temps de $n = 1600$.

C.7 Comparer avec Intel MKL

Pour comparer OpenBLAS et MKL, créer un second environnement `conda` dont NumPy et SciPy sont liés à MKL, puis relancer l'étape 2 (le backend doit afficher `mkl`) et l'étape 5 :

```
conda create -n mkl python=3.11 numpy scipy matplotlib threadpoolctl "libblas=*=*mkl"
conda activate mkl
python3 benchmark_blas.py
```

Reporter ensuite les deux séries de temps dans un tableau côte à côte (même machine, même nombre de threads, plusieurs répétitions). Le nombre de threads de MKL se règle avec `MKL_NUM_THREADS`.

C.8 Étape 6 : compiler le rapport

```
pdflatex main.tex
pdflatex main.tex       # 2e passage : table des matieres et references
```

La compilation demande une distribution LaTeX (TeX Live ou MiKTeX) avec le français pour `babel` (paquet `texlive-lang-french` sous Debian/Ubuntu ; MiKTeX l'installe automatiquement). Les figures du dossier `figures/` sont fournies déjà générées ; les chiffres que l'on peut recalculer sont ceux produits par `simulation_intra.py` et `benchmark_blas.py`.

C.9 Problèmes fréquents

Table C.2: Dépannage.

Symptôme	**Cause probable et solution**
`ModuleNotFoundError: vih_lapack`	Lancer `simulation_intra.py` depuis le dossier qui contient `vih_lapack.py`.
`ModuleNotFoundError: scipy` (ou autre)	Environnement non activé, ou `pip install` non exécuté.
`AssertionError: DGESV a echoue`	Matrice singulière : vérifier les paramètres du modèle (dictionnaire `PA`).
Temps du benchmark très variables	Fermer les autres programmes ; augmenter le nombre de répétitions `rep` dans `best_time`.
`Unknown option 'french'` à la compilation	Paquet de langue française de TeX manquant : l'installer (voir étape 6).

Dépôt du projet. https://github.com/JuniorB573/OCS-Groupe5